KINGDOM OF ESWATINI
Ministry of Labour and Social Security
Measurement and Testing Unit

System Design Document

SDS Test System

Online Self-Directed Search Career Assessment Platform

Document Version	1.1
Date	April 2026
Author	SDS Development Team
Client	Ministry of Labour and Social Security — Kingdom of Eswatini
Contact	Sibusiso Baartjies | sibusiso@datamatics.co.sz | 78026341
Classification	**Confidential**

Table of Contents

1. System Overview

1.1 Purpose

The Online Self-Directed Search (SDS) Test System is a comprehensive career assessment platform for the Ministry of Labour and Social Security of the Kingdom of Eswatini. It implements Holland's RIASEC model to help individuals identify suitable career paths based on their interests, abilities, and personality traits.

1.2 Key Business Requirements

Career Assessment	Implement the complete SDS questionnaire with 228-question RIASEC scoring
User Management	Support multiple user types with role-based access control (49 permissions, 13 modules)
Institution Integration	Link with local educational institutions and employers
Reporting & Analytics	Comprehensive analytics for administrators and counsellors
Data Protection	Compliance with Eswatini Data Protection Act 2022 and GDPR principles
Multi-language Support	English and siSwati language support throughout
Accessibility	WCAG 2.1 AA compliance for inclusive access

1.3 Stakeholders

Stakeholder	Role
Test Takers	High school students, university students, professionals seeking career guidance
Test Administrators	School counsellors and career guidance professionals managing student assessments
System Administrators	Ministry IT staff managing platform configuration, security, and maintenance
Institutions	Educational institutions and employers connected to the platform
Ministry of Labour	Government oversight body, primary system owner, and policy maker

2. Architecture

2.1 High-Level Architecture

The SDS Test System follows a three-tier web application architecture. The system context diagram below shows all external actors and the platform's interactions with each:

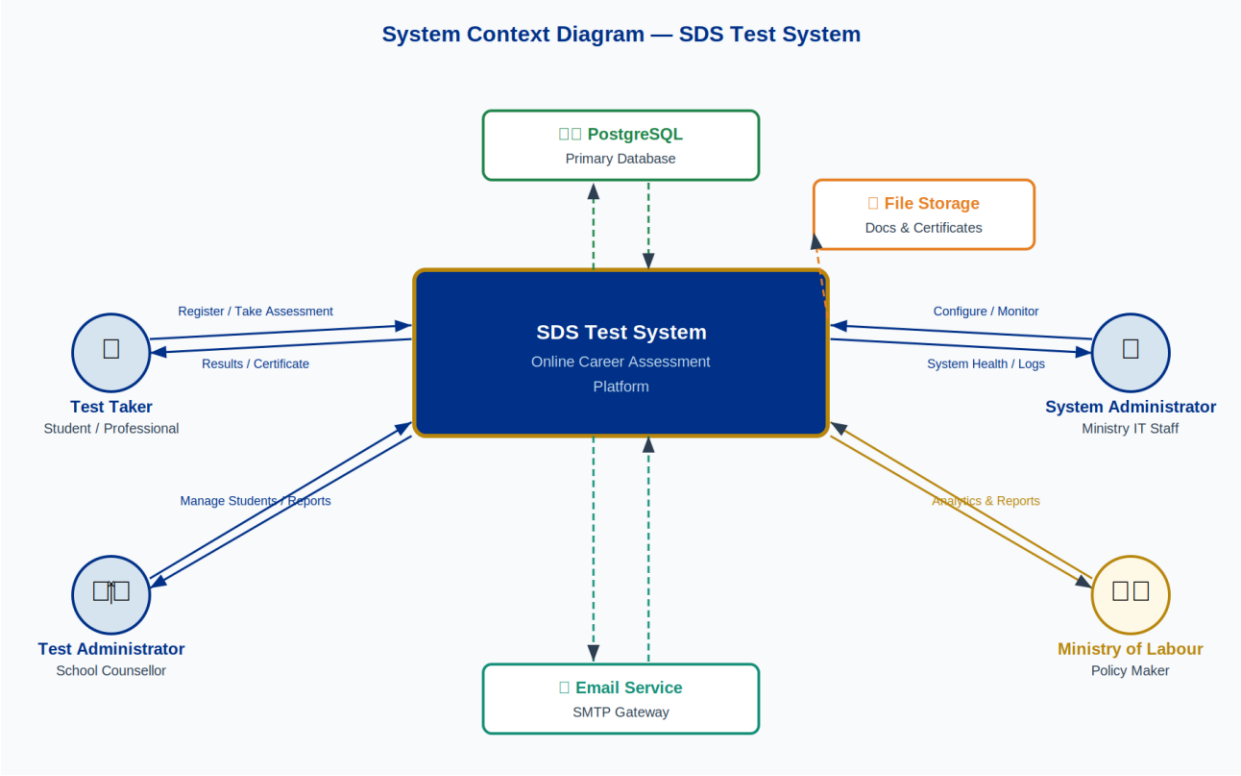

Figure 1: System Context Diagram — SDS Test System

2.2 Architectural Patterns

Pattern	Application in SDS
Model-View-Controller (MVC)	Backend follows MVC with an explicit service layer between controllers and models
Component-Based Architecture	React SPA built with Atomic Design (Atoms → Molecules → Organisms → Templates → Pages)
Repository Pattern	Data access abstracted via Sequalize ORM, isolating DB logic from business logic
Service Layer Pattern	Business logic separated into dedicated service classes away from HTTP controllers
Middleware Pattern	Express middleware chain handles cross-cutting concerns (auth, validation, rate limiting, logging)
Observer Pattern	Event-driven audit logging decoupled from core business operations via emitted domain events

2.3 Design Principles

Principle	Description
Separation of Concerns	Clear enforced boundaries between presentation, business logic, and data layers
Single Responsibility	Each component, service, and module has exactly one well-defined purpose
Dependency Inversion	High-level business modules do not depend directly on low-level infrastructure
Open/Closed Principle	System components are open for extension via configuration without modifying core code
Don't Repeat Yourself (DRY)	Shared utilities, reusable components, and centralized validation prevent code duplication

3. Technology Stack

3.1 Frontend Stack

Technology	Version	Purpose
React	18	SPA framework with hooks, concurrent rendering, and Suspense
React Router	v6	Client-side routing with lazy loading and route-based code splitting
Tailwind CSS	3+	Utility-first CSS framework with Ministry design system tokens
Lucida React	—	Consistent icon library across all UI components
Axios	—	HTTP client with interceptors, token refresh, and retry logic
React Query	—	Server state management, background prefetching, and client-side cache
React Hook Form	—	Performant form handling with built-in validation
React PDF	—	Client-side PDF generation for certificate previews

3.2 Backend Stack

Technology	Version	Purpose
Node.js	18+	JavaScript runtime with ES2022 and built-in fetch API
Express.js	4	Lightweight web application framework with middleware support
Sequalize	6	ORM providing model definitions, migrations, and query abstraction
PostgreSQL	14+	Primary RDBMS with UUID primary keys and JSONB support
JWT	—	Stateless authentication via RS256-signed access and refresh tokens
bcrypt	—	Password hashing with 10-round salt
Winston	—	Structured JSON logging with configurable transports
Multer	—	Multipart form data and file upload handling with MIME validation
PDFKit	—	Server-side PDF generation for certificates and formal reports

3.3 Development & Quality Tools

Tool	Purpose
Jest	Unit and integration testing for both frontend and backend
Cypress	End-to-end automated browser testing for critical user journeys
ESLint + Prettier	Code quality enforcement across the entire codebase
Husky	Git pre-commit hooks ensuring linting and tests pass before any commit
Artillery	Load and performance testing — 500+ concurrent user simulation
nodemon	Development server with automatic restart on file changes

4. System Components

4.1 Component Overview

The component diagram below shows how the three major software packages (Frontend, Backend API, and External Services) interact through well-defined interfaces:

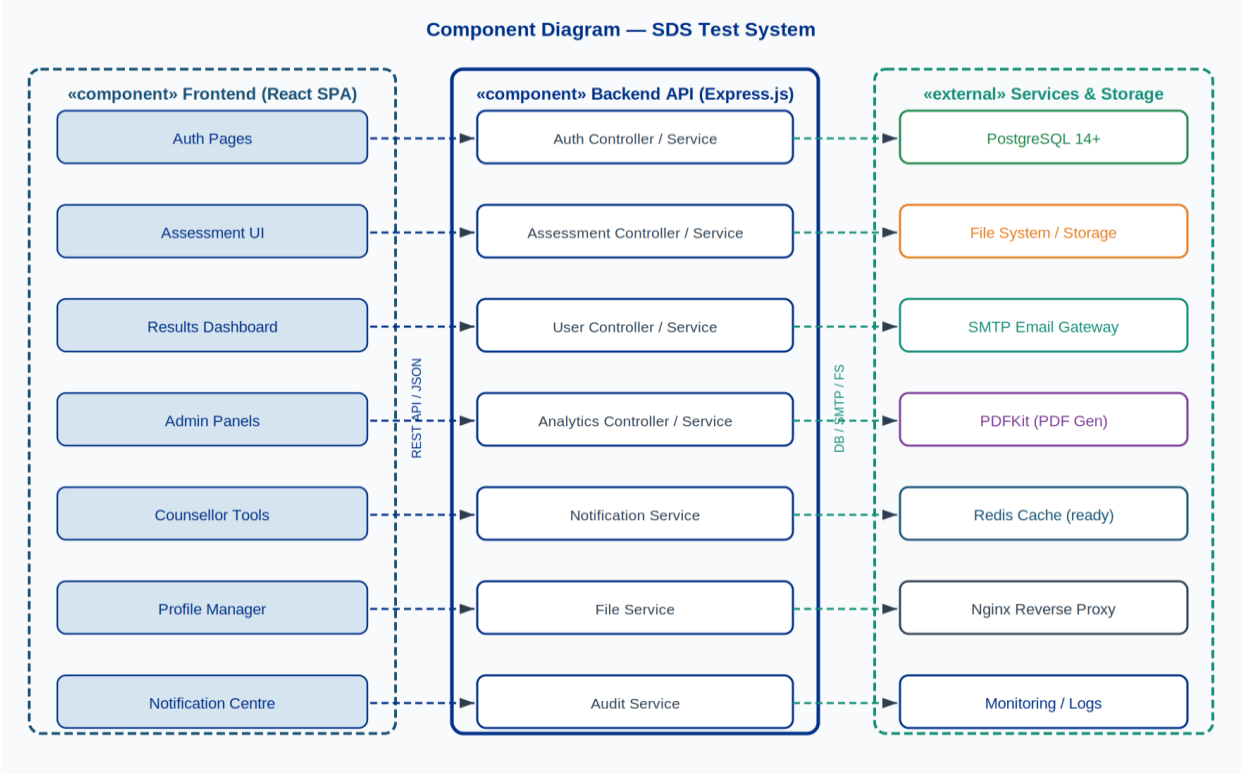

Figure 2: Component Diagram — SDS Test System

4.2 Frontend Structure

```
src/
├── components/          # Reusable UI components
│   ├── ui/              # Atoms: Button, Input, Modal, Badge
│   ├── layout/          # Organisms: Header, Sidebar, Footer
│   └── common/          # Shared business-level components
├── features/            # Feature-scoped component trees
│   ├── admin/           # System administration panels
│   ├── counselor/       # Test administrator / counsellor tools
│   ├── assessment/      # SDS assessment engine components
│   └── analytics/       # Reporting and analytics dashboards
├── pages/               # Route-level page components
├── services/            # Axios API service layer
├── hooks/               # Custom reusable React hooks
├── context/             # React context providers (Auth, Theme, i18n)
└── utils/               # Pure utility and helper functions
```

4.3 Backend Structure

```
src/
├── controllers/        # HTTP handlers: parse input, call service, format response
├── services/           # Business logic: orchestrate operations and transactions
├── models/             # Sequelize schemas, associations, query scopes
├── routes/             # Express route definitions with middleware chains
├── middleware/         # Auth, RBAC, validation, rate limiting, logging
├── utils/              # Shared helpers: date, encryption, file, pagination
├── validations/        # Joi input validation schemas per endpoint
└── templates/          # Handlebars email templates and PDFKit layouts
```

4.4 Core Backend Services

Service	Responsibilities
Authentication Service	JWT issuance/validation, refresh token rotation, bcrypt hashing, OTP management
Assessment Service	Session creation, question sequencing, answer persistence, RIASEC scoring, Holland code generation
User Service	User CRUD, profile updates, role assignment, bulk CSV import, privacy controls
Analytics Service	Data aggregation, report generation, dashboard metrics, export formatting
Notification Service	SMTP email dispatch, in-app notification creation, template rendering, delivery tracking
File Service	Upload handling, MIME validation, secure storage, document retrieval
Audit Service	Event capture, immutable log persistence, audit report generation

5. Data Model

5.1 Class Diagram

The class diagram below shows the core domain objects, their attributes, key methods, and the relationships between them:

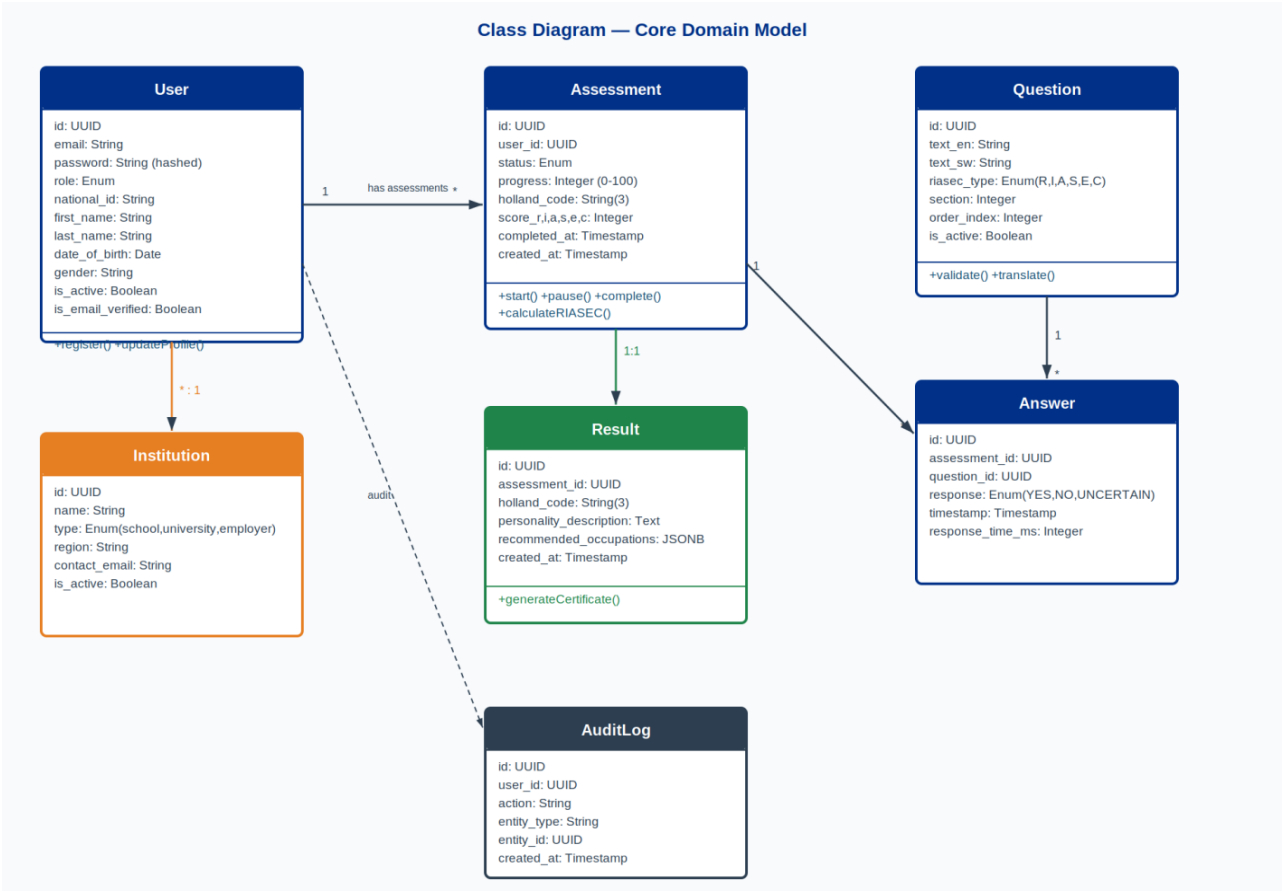

Figure 3: Class Diagram — Core Domain Model

5.2 Entity Groups

5.2.1 User Management

Entity	Description
Users	Central user entity with role, national ID, institutional affiliation, and account status
Permissions	49 granular permissions across 13 functional modules
UserPermissions	Junction table resolving which users hold which permissions
EducationLevels	Lookup for educational attainment levels used in profiles and recommendations
Institutions	Schools, universities, and employers with type, region, and contact data
Occupations	Career occupations with Holland codes and education requirements

5.2.2 Assessment System

Entity	Description
Assessments	Test sessions with status, progress %, RIASEC raw scores, and Holland code
Questions	SDS questionnaire items with RIASEC dimension and section classification
Answers	User responses (YES/NO) with timestamps and response duration
Results	Processed outcomes including personality description and occupation recommendations
Certificates	Generated PDF completion certificates with unique verifiable identifiers

5.2.3 Content & Audit

Entity	Description
Subjects	Academic subjects with Holland code mappings for career recommendations
Courses	Educational courses linked to institutions with entry requirements
AuditLogs	Immutable log of all system events with actor, action, and timestamp
UserQualifications	User-uploaded qualification document records with metadata

6. API Design

6.1 RESTful API Structure

All endpoints are versioned under /api/v1/ and organised by resource domain:

```
/api/v1/
├── auth/              # login, register, refresh, logout, verify-email, reset-password
├── assessments/       # start, answer, progress, pause, complete, results
├── users/             # CRUD, bulk-import, role-assignment (admin only)
├── admin/             # system-config, audit-logs, system-health
├── counselor/         # student-lists, class-import, institution-reports
├── institutions/      # schools, universities, employers — CRUD + import
├── occupations/       # Holland code lookup, career recommendations
├── analytics/         # dashboards, custom reports, exports
├── notifications/     # list, read, preferences, mark-all-read
├── certificates/      # generate, download, verify
└── qualifications/    # upload, view, delete
```

6.2 Standard Response Envelope

```
{
  "status":  "success | error",
  "data":    { ... },             // Resource payload on success
  "message": "Human-readable message",
  "errors":  [ ... ]              // Field-level validation errors only
}
```

6.3 HTTP Status Code Reference

Code	Meaning	When Used
200	OK	Successful GET, PATCH, or PUT
201	Created	Successful POST creating a new resource
204	No Content	Successful DELETE with no response body
400	Bad Request	Input validation failure — errors array populated
401	Unauthorised	Missing or expired authentication token
403	Forbidden	Valid token but insufficient permissions
404	Not Found	Requested resource does not exist
409	Conflict	Duplicate national ID, email, or other unique constraint
429	Too Many Requests	Rate limit exceeded — Retry-After header included
500	Internal Server Error	Unexpected server error — generic message, full detail logged

7. Security Architecture

7.1 Defence in Depth

Layer	Controls
1. Network Security	HTTPS/TLS 1.3 everywhere; CORS whitelist policy; Content Security Policy headers via Helmet.js
2. Application Security	Input validation (Joi schemas); SQL injection prevention (parameterised Sequelize queries); XSS protection
3. Authentication Security	JWT RS256 signing; bcrypt hashing; httpOnly refresh token cookies; OTP email verification
4. Authorisation Security	RBAC with 49 granular permissions; checks enforced at endpoint and data layer
5. Data Security	AES-256 at rest for sensitive fields; TLS 1.3 in transit; PII minimisation by design
6. Audit Security	Immutable audit log; real-time anomaly alerting; regular vulnerability scans

7.2 Security Controls

Control	Library / Method	Purpose
Security Headers	Helmet.js	CSP, HSTS (max-age 1yr), X-Frame-Options, Referrer-Policy
Rate Limiting	express-rate-limit	100 req/min global; 10/min on auth endpoints; lockout after 5 failed logins
Input Validation	Joi	Schema-based validation of all bodies, params, and query strings
SQL Injection Prevention	Sequelize ORM	Parameterised queries; no raw SQL string interpolation
XSS Protection	Input sanitisation	Server-side sanitisation before persistence; output encoding on API responses
CSRF Protection	SameSite cookies	SameSite=Strict on refresh token cookie; state-changing form protection

8. Authentication & Authorisation

8.1 Authentication Flow

The system uses stateless JWT authentication. Access tokens are stored in application memory only (never localStorage). The middleware stack diagram in Appendix G illustrates the request processing chain.

```
1. User submits email + password
2. Backend validates credentials against bcrypt hash
3. JWT access token (RS256, 1hr TTL) + refresh token issued
4. Access token stored in React app memory — NOT localStorage
5. Refresh token set as httpOnly, Secure, SameSite=Strict cookie
6. Axios interceptor auto-calls /auth/refresh when access token expires
7. Logout: refresh token invalidated server-side; cookie cleared
```

8.2 JWT Token Payload

```
{
  "sub":         "user-uuid",
  "email":       "user@example.com",
  "role":        "Test Taker | Test Administrator | System Administrator",
  "permissions": ["assessments.view", "results.view", "certificates.download"],
  "iat":         1640995200,   // Issued at (Unix)
  "exp":         1640998800    // Expires in 1 hour
}
```

8.3 Role-Based Access Control (RBAC)

Role	Permissions	Scope
System Administrator	49 permissions	Full system access — all 13 modules, all operations
Test Administrator	16 permissions	Student and test management within assigned institution
Test Taker	Basic permissions	Own assessment, results, profile, and certificates only

8.4 Permission Matrix (13 Modules, 49 Permissions)

Module	Permissions
users	`view, create, update, delete, export`
institutions	`view, create, update, delete, export, import`
questions	`view, create, update, delete, import, export`
occupations	`view, create, update, delete, import, export`
subjects	`view, create, update, delete, import, export`
assessments	`view, create, update, delete, export`
results	`view, export`
analytics	`view, export`
audit	`view`
notifications	`view, manage`

certificates	`view, generate, download`
permissions	`view, manage`
test_takers	`manage`

9. Frontend Architecture

9.1 Component Architecture

Pattern	Application
Atomic Design	Atoms → Molecules → Organisms → Templates → Pages hierarchy enforced via directory structure
Component Composition	Complex UIs built by composing single-purpose components via props and render props
State Management	Local state (useState) for UI; React Query for all server state and caching
Error Boundaries	Feature-level error boundaries prevent full-page crashes from isolated component failures
Code Splitting	React.lazy() + Suspense for route-level lazy loading; vendor chunk separation

9.2 State Management

```
// 1. Local UI state — ephemeral, component-scoped
const [isOpen, setIsOpen] = useState(false);

// 2. Server state — React Query handles fetch, cache, background sync
const { data, isLoading, error } = useQuery({
  queryKey: ['assessment', sessionId],
  queryFn:  () => assessmentService.getSession(sessionId),
  staleTime: 30 * 1000   // 30-second cache for active assessment
});

// 3. Global auth state — React Context (no external store needed)
const { user, permissions, logout } = useAuth();
```

9.3 Performance Optimisations

Optimisation	Technique
Memoisation	React.memo for pure components; useMemo for expensive derived data; useCallback for stable handlers
Virtual Scrolling	react-window windowed lists for student tables and question banks > 100 rows
Image Optimisation	WebP with JPEG fallback; lazy loading via Intersection Observer
Bundle Optimisation	Tree shaking; dynamic imports per route; vendor/commons chunk splitting
Caching Strategy	React Query client cache (configurable staleTime); HTTP ETag/Last-Modified headers; service worker for static assets

10. Backend Architecture

10.1 Service Layer Pattern

Every HTTP request flows through the following strict layer sequence. The middleware stack diagram in Section 10.2 illustrates the detailed request processing pipeline.

```
HTTP Request (inbound)
    |
    ▼  Middleware chain (see Section 10.2)
Controller   ← Parse HTTP; validate input; call service; format response
    |
    ▼
Service      ← Business logic; orchestrate operations; manage transactions
    |
    ▼
Model        ← Sequelize schema; query scopes; association definitions
    |
    ▼
PostgreSQL   ← Persistent data storage
```

10.2 Express Middleware Stack

All requests pass through the following middleware chain in strict order:

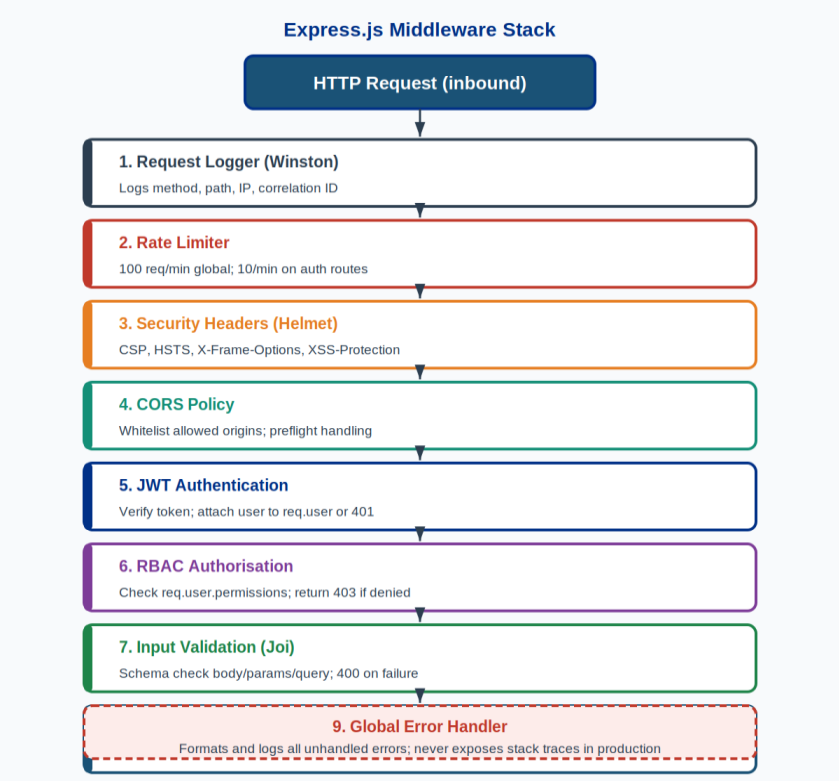

Figure 4: Express.js Middleware Stack

10.3 Layer Responsibilities

Layer	Responsibilities

Controller	HTTP request/response handling; input parsing; service delegation; error propagation; response formatting
Service	Business logic; data transformation; external service calls; complex query orchestration; transaction management
Model	Sequelize schema definition; data validation rules; association definitions; reusable query scopes and instance methods

11. Database Design

11.1 Entity-Relationship Diagram

The complete database schema with all tables, primary keys, foreign keys, and cardinality:

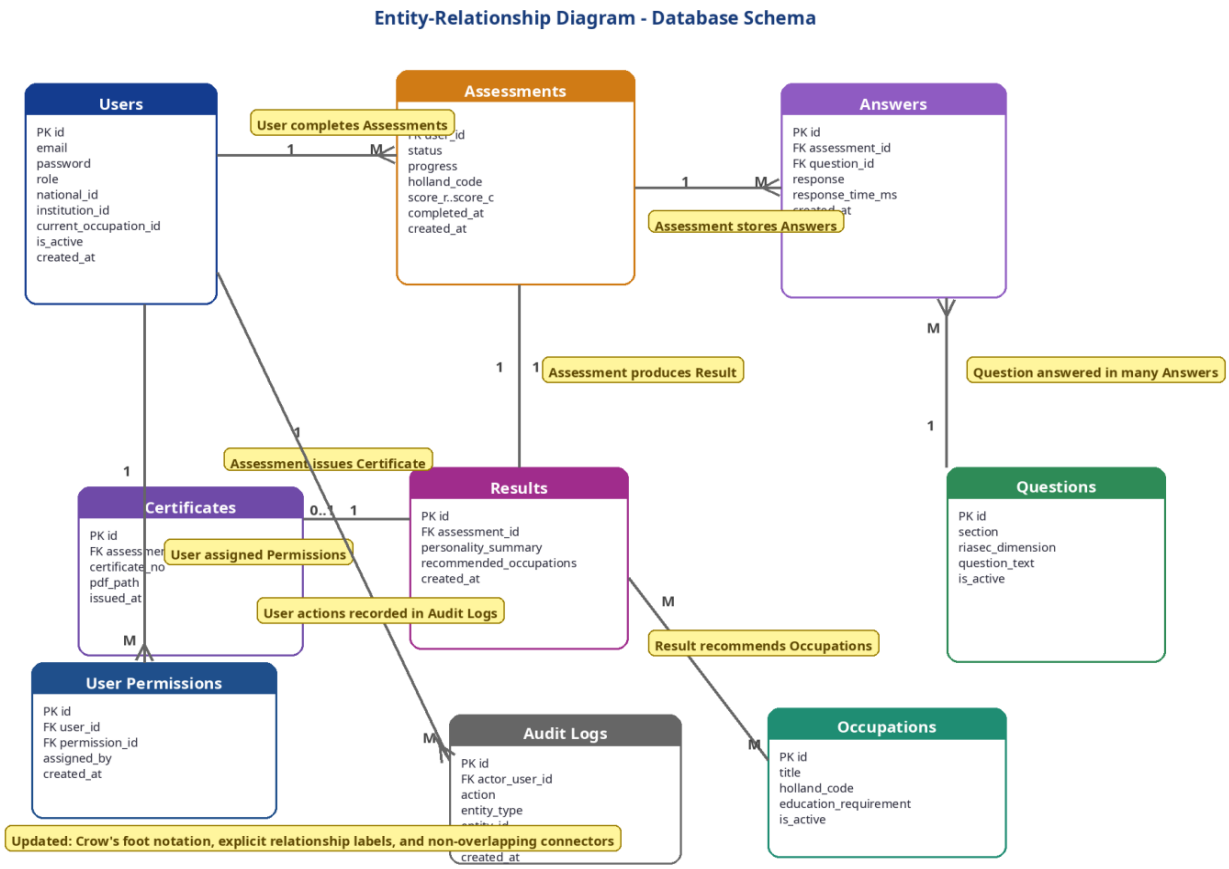

Figure 5: Entity-Relationship Diagram — Full Database Schema

11.2 Users Table Schema

```
CREATE TABLE users (
  id                    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email                 VARCHAR(255) UNIQUE NOT NULL,
  password              VARCHAR(255) NOT NULL,          -- bcrypt hash
  role                  VARCHAR(50)  NOT NULL,          -- enum via application
  national_id           VARCHAR(13)  UNIQUE,            -- 13-digit Eswatini ID
  first_name            VARCHAR(100),
  last_name             VARCHAR(100),
  date_of_birth         DATE,                           -- auto-extracted from
national_id
  gender                VARCHAR(20),                    -- auto-extracted from
national_id
  institution_id        UUID REFERENCES institutions(id) ON DELETE SET NULL,
  current_occupation_id UUID REFERENCES occupations(id)  ON DELETE SET NULL,
  is_active             BOOLEAN DEFAULT true,
  is_email_verified     BOOLEAN DEFAULT false,
  created_at            TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at            TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

11.3 Assessments Table Schema

```
CREATE TABLE assessments (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
  status       VARCHAR(20) NOT NULL,   -- in_progress | completed | paused | expired |
abandoned
  progress     INTEGER DEFAULT 0,      -- 0 to 100 (percentage of 228 questions answered)
  holland_code CHAR(3),                -- generated on completion e.g. 'RIA'
  score_r      INTEGER DEFAULT 0,      -- Realistic raw score
  score_i      INTEGER DEFAULT 0,      -- Investigative raw score
  score_a      INTEGER DEFAULT 0,      -- Artistic raw score
  score_s      INTEGER DEFAULT 0,      -- Social raw score
  score_e      INTEGER DEFAULT 0,      -- Enterprising raw score
  score_c      INTEGER DEFAULT 0,      -- Conventional raw score
  completed_at TIMESTAMP,
  created_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at   TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

11.4 Database Performance Optimisations

Optimisation	Implementation
Indexing Strategy	Composite indexes on (user_id, created_at); GIN index on JSONB results.recommended_occupations; index on assessments.status
Query Optimisation	Sequelize eager loading (include) prevents N+1; selective field projection reduces payload size
Connection Pooling	PgBouncer manages connection pool; pool size = CPU cores × 2; max 100 connections per instance
Read Replicas	Analytics and report queries routed to streaming replica to offload primary write database
Table Partitioning	Range partitioning on audit_logs.created_at and answers.timestamp for large-scale queries

12. Integration Patterns

12.1 Data Flow Diagram

The Level 1 DFD below shows how data flows between system processes, external actors, and data stores:

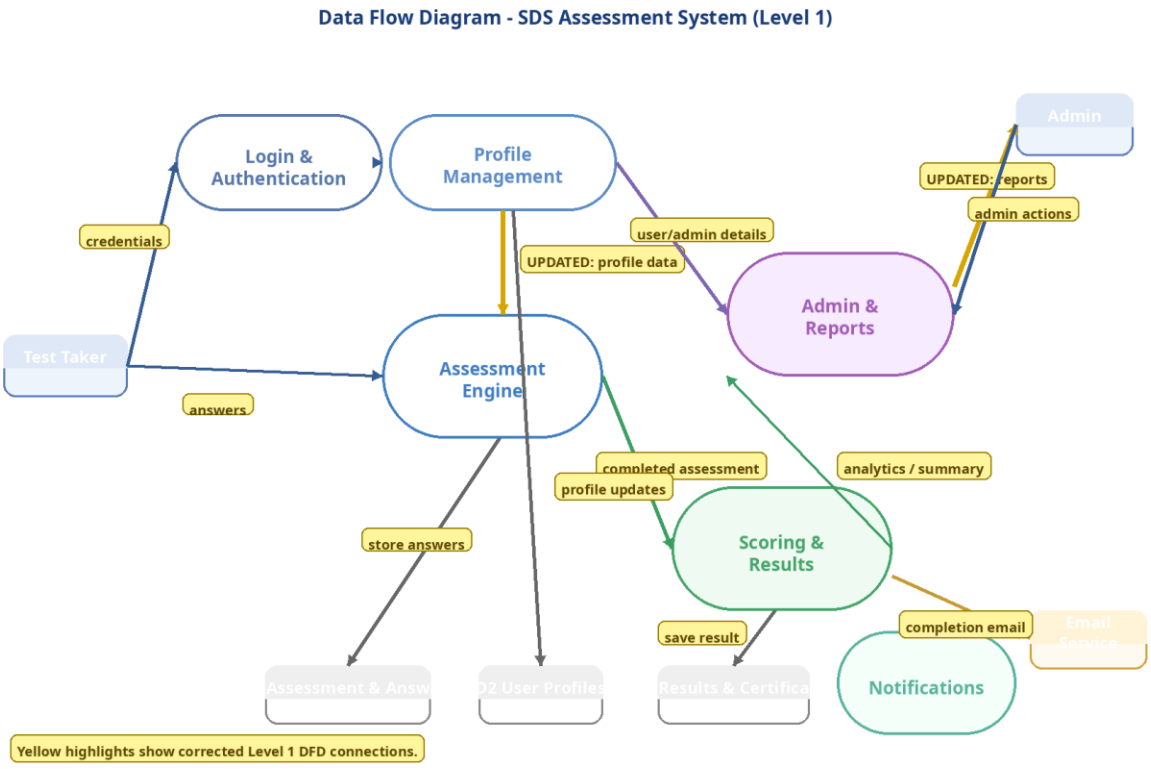

Figure 6: Data Flow Diagram — SDS Assessment System (Level 1)

12.2 External Service Integrations

Integration	Technology	Purpose
Email Service	SMTP (Nodemailer)	Transactional emails: verification, password reset, assessment completion, admin alerts
File Storage	Local filesystem	Document uploads and certificate PDFs; S3-compatible cloud migration path available
PDF Generation	PDFKit (server)	Formal certificate and report PDF generation with Ministry branding and unique codes
CSV Processing	Stream processing	Memory-efficient stream-based parsing for large bulk student import files
Analytics	PostgreSQL aggregations	Window functions and CTEs for dashboard metrics; Power BI export readiness

12.3 Design Patterns Applied

Pattern	Where Applied
Repository Pattern	Sequelize model layer abstracts all database access from service and controller layers
Service Locator	Dependency injection container manages service instantiation and lifecycle
Observer Pattern	Audit service subscribes to domain events emitted by services — decouples logging from business logic
Strategy Pattern	Different authentication strategies (JWT, OTP) implement a common validation interface
Factory Pattern	Database connection factory manages pool creation; model factory simplifies test fixture setup

13. Performance Considerations

13.1 Frontend Performance Targets

Area	Approach	Target
Initial Load	Route-level code splitting; CDN delivery of static assets	< 3 sec on 2 Mbps
Assessment UX	Questions pre-fetched in background; local state for instant UI updates	< 1 sec per question
Data Tables	Virtual scrolling for lists > 100 rows; paginated API responses	Smooth 60 fps scroll
Image Delivery	WebP with JPEG fallback; lazy loading via Intersection Observer	< 1 sec per image
Bundle Size	Tree shaking; dynamic imports; vendor chunk separation	< 200 KB initial JS

13.2 Backend Performance Targets

Area	Approach	Target
API Response	Optimised queries; selective field projection; N+1 prevention via eager loading	< 2 sec (p95)
Database Queries	Composite indexes; read replicas for analytics; query plan analysis	< 500 ms (p95)
File Uploads	Streaming multipart parsing; async storage; immediate 202 Accepted response	Non-blocking
PDF Generation	Background job queue for large reports; inline generation for certificates	< 5 sec
Caching	Redis-ready layer; React Query client cache; HTTP ETag headers	40% DB load reduction

14. Scalability

14.1 Horizontal Scaling

Component	Horizontal Scaling Approach
Application Servers	Stateless JWT authentication enables multiple Node.js instances behind Nginx load balancer
Database	Streaming read replica for analytics; Citus or sharding when primary DB reaches capacity limits
File Storage	Local filesystem in Phase 1; S3-compatible object storage (MinIO or AWS S3) in Phase 2
Session Management	No server-side session state — JWT tokens eliminate sticky session requirements completely
Background Jobs	Bull queue with Redis enables distributed job processing across multiple worker instances

14.2 Capacity Planning

Concurrent Users	500+ in initial deployment; architecture horizontally scales to 5,000+ without redesign
Database Scale	Millions of records supported; partitioning applied at 10M+ row thresholds per table
File Storage	Per-user and per-institution quotas configurable; cloud storage migration available
Network	Optimised payloads and HTTP/2 compression ensure full functionality on 2 Mbps connections
Annual Growth	Designed to accommodate 100% annual growth in user base for the first three years

15. Deployment Architecture

15.1 Deployment Diagram

The deployment diagram below shows the physical infrastructure, server roles, and network topology:

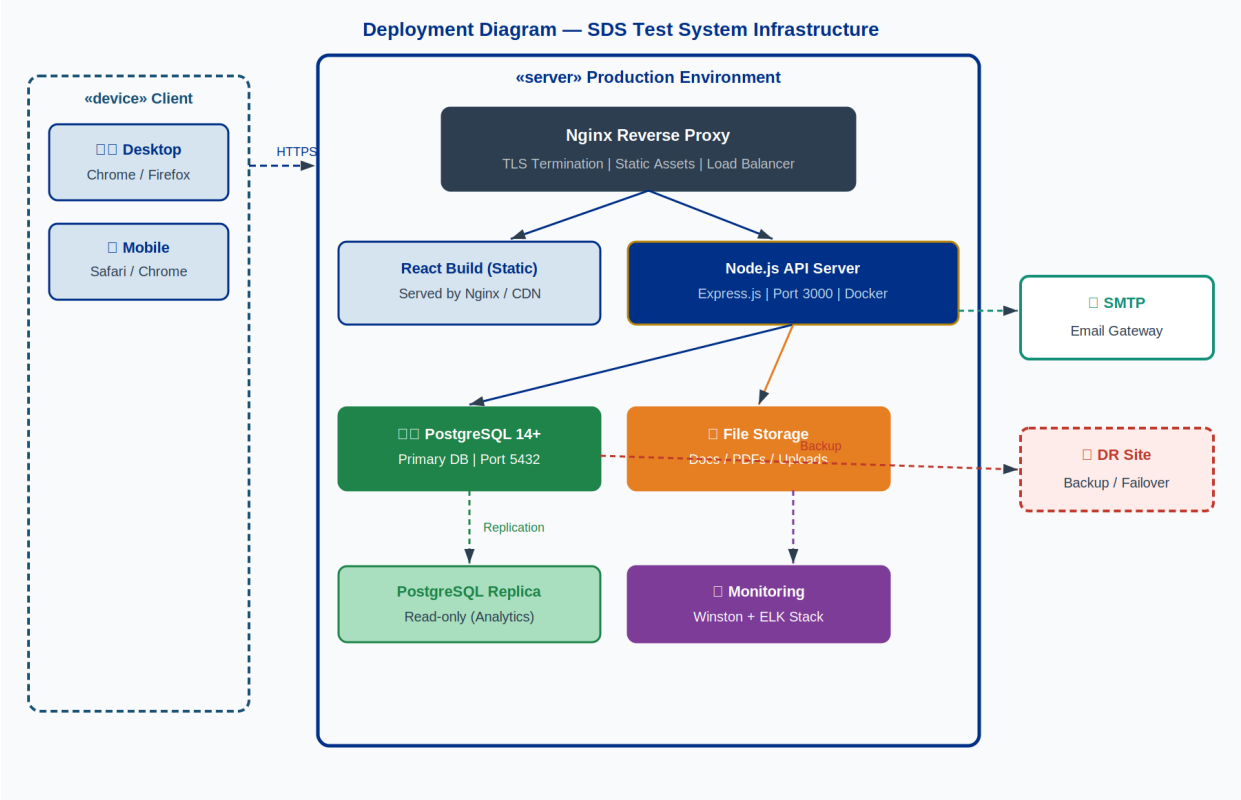

Figure 7: Deployment Diagram — Production Infrastructure

15.2 Environment Strategy

Environment	Purpose	Key Differences from Production
Development	Local developer workstations with hot-reload	Mock services; debug logging; relaxed CORS; no TLS
Staging	Production-like integration and UAT environment	Real services; anonymised data; full security controls
Production	Live system serving Ministry users	Full HA; real-time monitoring; automated backups; TLS
Disaster Recovery	Warm standby for business continuity	Tested failover; RTO < 4 hours; RPO < 1 hour

15.3 Deployment Pipeline

```
1. Developer opens Pull Request (feature → main branch)
2. GitHub Actions triggers automatically:
   a. ESLint + Prettier code quality checks
   b. Jest unit and integration test suite
   c. React + Node.js build verification
3. Code review and approval by senior developer
```

```
4. Merge to main → Docker image built and tagged with commit SHA
5. Trivy / Snyk security vulnerability scan on Docker image
6. Auto-deploy to Staging environment
7. Cypress E2E test suite against Staging
8. Manual UAT sign-off by Ministry representative
9. Blue-green production deployment (zero downtime)
10. Post-deployment health checks and smoke tests
11. Rollback available within 15 min if health checks fail
```

15.4 Kubernetes Orchestration, Infrastructure as Code & CI/CD

15.4.1 Kubernetes Deployment

The production SDS Test System shall be deployed on Kubernetes using container images built from the project Dockerfiles. All workloads shall be defined as declarative Kubernetes manifests or packaged as Helm charts and version-controlled alongside the application source code. The deployment shall cover the React frontend (served via Nginx), the Node.js API server, the PostgreSQL primary database and read replica, and the supporting services (Redis cache, monitoring stack).

Kubernetes Capability	Design Detail
Workload manifests	Kubernetes Deployment, Service, ConfigMap, and Secret manifests (or Helm chart equivalents) delivered as part of the source code repository.
Horizontal auto-scaling	HorizontalPodAutoscaler (HPA) configured on the API server deployment. Scales on CPU utilisation (target 70%) and optionally on custom metrics (request rate). Minimum 2 replicas; maximum configurable per environment.
Self-healing	Liveness and readiness probes defined for all containers. Kubernetes automatically restarts failed containers and reschedules pods from unhealthy nodes. Rolling update strategy ensures zero-downtime deployments.
Namespace isolation	Separate Kubernetes namespaces for staging and production environments. Network policies restrict cross-namespace traffic. RBAC limits cluster access to authorised personnel only.

15.4.2 Infrastructure as Code (IaC)

All infrastructure required to build, deploy, and operate the SDS Test System shall be defined and delivered as Terraform templates (or an approved equivalent IaC toolchain). IaC artefacts shall be version-controlled, peer-reviewed, and stored in the project repository. The following infrastructure components shall be covered:

IaC Module	Scope
Compute & Kubernetes cluster	Node pool definitions, cluster configuration, and namespace provisioning.
Database	PostgreSQL instance provisioning, backup configuration, and read replica setup.
Networking & security	Firewall rules, load balancer, TLS certificate provisioning, DNS records, and network policy definitions.
Storage & container registry	Persistent volume configuration, object storage for file uploads, and container image registry provisioning.
Monitoring stack	Log aggregation, metrics collection, and alerting infrastructure provisioned as code.

15.4.3 CI/CD Pipeline Design

The CI/CD pipeline defined in Section 15.3 is implemented using GitHub Actions. Pipeline definition files are stored in .github/workflows/ within the source repository and form a required delivery artefact. The pipeline stages and their design responsibilities are as follows:

Pipeline Stage	Tools / Actions	Pass Criteria
Code Quality	ESLint, Prettier	Zero lint errors; consistent formatting enforced
Automated Tests	Jest (unit + integration), Cypress (E2E)	All tests pass; coverage targets met (80% FE, 90% BE)
Build	Docker build; image tagged with commit SHA; pushed to container registry	Image built and pushed successfully; build log retained
Security Scan	Trivy / Snyk container and dependency vulnerability scan	No Critical or High CVEs; scan report retained as pipeline artefact
Staging Deployment	kubectl apply / helm upgrade to staging Kubernetes namespace; Cypress E2E suite executed against staging	Staging health checks pass; all E2E tests pass; deployment log retained

16. Monitoring & Logging

16.1 Logging Strategy

Log Type	Format	Content
Application Logs	JSON (Winston)	Request method, path, status, response time, user ID, correlation ID
Audit Logs	JSON (immutable)	Actor, action, entity type, entity ID, before/after state, timestamp, IP
Error Logs	JSON (Winston)	Error type, message, stack trace, request context, severity level
Performance Logs	JSON (Winston)	Endpoint, duration, DB query count, cache hit/miss ratio
Security Logs	JSON (Winston)	Auth events, permission denials, rate limit hits, suspicious IP patterns

16.2 Alerting Thresholds

Alert Type	Threshold	Channel
Error Rate	Error rate > 1% over 5 minutes	PagerDuty + Slack #ops
Performance	p95 response time > 3 seconds	Slack #ops
Resource	CPU or Memory > 85% for 10 minutes	Slack #ops + email
Security	5+ failed logins from single IP in 1 minute	PagerDuty + Security team
Database	Replication lag > 30 seconds	PagerDuty + DBA
Business	Zero assessment completions in 2 hours during business hours	Slack #ministry-ops

17. Error Handling

17.1 Error Classification

Error Type	HTTP Code	Handling Approach
Validation Errors	400	Joi schema errors mapped to field-level error objects in the errors array
Authentication Errors	401	JWT verification failure or expired token — client must re-authenticate
Authorisation Errors	403	Valid user but missing required permission — permission name included
Not Found Errors	404	Resource does not exist — resource type and ID in message
Conflict Errors	409	Duplicate unique constraint — field name included in response
Server Errors	500	Unexpected internal error — full trace logged; generic message to client
Network Errors	—	Axios retry (3 attempts, exponential backoff) before surfacing to user

17.2 Frontend Error Boundaries

Mechanism	Implementation
Error Boundaries	React error boundaries at feature module level catch render errors gracefully
Global Error Handler	Axios interceptor translates API and network errors to user-friendly messages
Fallback UI	Every error boundary renders a fallback card with retry action and support contact
Error Reporting	Unhandled errors reported to monitoring service with user context and component stack trace

18. Testing Strategy

18.1 Testing Pyramid

Level	Coverage	Framework	Scope
Unit Tests (70%)	80%+ FE; 90%+ BE	Jest	Services, utilities, RIASEC scoring algorithm, React hooks
Integration Tests (20%)	API endpoints	Jest + Supertest	Full request-response cycle against test database
E2E Tests (10%)	Critical journeys	Cypress	Registration, assessment completion, results, certificate download

18.2 Test Coverage Targets

Scope	Target	Rationale
Frontend overall	>80%	Balance between coverage confidence and maintenance overhead
Backend overall	>90%	Higher target due to critical business logic concentration
Authentication paths	100%	Zero tolerance for security regressions
RIASEC scoring algorithm	100%	Core product value — accuracy is non-negotiable
Critical E2E journeys	100%	All primary user journeys covered in automated E2E suite

19. Data Migration Strategy

19.1 Migration Management

Concern	Approach
Schema Versioning	Sequelize migrations with up() and down() methods for full reversibility
Data Seeding	Separate seed scripts populate reference data (228 questions, occupations, education levels)
Rollback Strategy	Every migration has a tested down() method; rollback executable within minutes
Staging Validation	All migrations run on staging with production-equivalent anonymised data before prod
Documentation	Each migration file includes a header comment: change, business reason, data assumptions

19.2 Migration Execution Process

```
1. Write migration with up() and down() methods; include rollback test
2. Peer review by second developer
3. Run on Staging with anonymised production-equivalent data
4. Verify application functionality post-migration on Staging
5. Test rollback procedure on Staging
6. Take verified backup of production database
7. Schedule within maintenance window; apply with monitoring active
8. Execute post-migration smoke tests
9. Confirm success; monitor error rates and slow queries for 24 hours
```

20. Compliance & Governance

20.1 Data Protection

Requirement	Implementation
Eswatini Data Protection Act 2022	All data processing, storage, and transfer compliant with local legislation
GDPR Principles — Privacy by Design	Data protection built into architecture from inception; not retrofitted
Data Minimisation	Only fields strictly necessary for system function are collected
Purpose Limitation	Data collected for career assessment not used for unrelated purposes
Storage Limitation	Configurable retention policies auto-anonymise or delete data after defined periods
Right to Erasure	Complete user data deletion with cascade rules; audit log entry retained per legal requirement

20.2 Audit Requirements

Requirement	Implementation
Audit Trail	All user actions, data changes, and admin operations logged to immutable audit_logs table
Data Integrity	Database constraints, triggers, and application-level validation maintain consistent state
Access Control Audit	RBAC permission checks logged; access denials recorded with user, resource, and timestamp
Change Management	All production changes require PR review, staging validation, and documented approval
Incident Response	Documented runbook for security incidents; escalation path to Ministry IT security officer

21. Design Decisions & Rationale

This section documents the key architectural decisions made during design, the alternatives considered, and the rationale for the chosen approach.

Decision	Chosen Approach	Alternatives Considered	Rationale
Authentication mechanism	JWT stateless tokens (RS256)	Session-based (server-side store), OAuth2 only	JWT eliminates server-side session state, enabling horizontal scaling. RS256 asymmetric signing is more secure than HS256 for distributed systems.
Token storage	Access token in memory; refresh in httpOnly cookie	Both tokens in localStorage	localStorage is accessible to JavaScript, making it vulnerable to XSS attacks. httpOnly cookies prevent JS access entirely.
Frontend framework	React 18 with React Query	Angular, Vue, Next.js SSR	React's ecosystem maturity and team familiarity; React Query provides excellent server state management without Redux overhead. SSR not required as SEO is not a primary concern.
Database	PostgreSQL 14+ with Sequelize ORM	MySQL, MongoDB, Firebase	PostgreSQL's JSONB support offers flexibility for recommendation payloads; ACID compliance critical for assessment integrity; mature ecosystem; better analytical query support than MySQL.
PDF generation	PDFKit (server-side)	Puppeteer/headless Chrome, client-side React PDF	PDFKit produces consistent output without browser dependency; server-side generation ensures certificate authenticity and reduces client resource usage.
Scoring algorithm	Pure SQL aggregation + Node.js calculation	ML model, third-party scoring API	Holland's RIASEC is deterministic — no ML needed. Pure calculation is transparent, auditable, and zero-latency. Avoids vendor dependency for core functionality.
RIASEC scoring	YES=1, NO/UNCERTAIN=0	Likert scale (YES=2, UNCERTAIN=1, NO=0)	Standard Holland self-directed search methodology uses binary YES/UNCERTAIN/NO with only YES responses counted. Maintains scientific validity and comparability.
Deployment approach	Containerised (Docker) + Nginx	Bare metal, PaaS (Heroku)	Docker ensures environment consistency; Nginx handles TLS termination and load balancing. More control than PaaS; less operational overhead than bare metal.

22. Glossary

SDS	Self-Directed Search — Holland's standardised career interest assessment tool
RIASEC	Holland's six personality/interest types: Realistic, Investigative, Artistic, Social, Enterprising, Conventional
Holland Code	3-character code derived from the three highest-scoring RIASEC dimensions (e.g. 'RIA')
JWT	JSON Web Token — compact, URL-safe token for stateless authentication
RS256	RSA Signature with SHA-256 — asymmetric signing algorithm used for JWT tokens
ORM	Object-Relational Mapper — Sequelize abstracts SQL queries into JavaScript model methods
RBAC	Role-Based Access Control — permissions granted to roles, not directly to individuals
WCAG	Web Content Accessibility Guidelines — W3C standard for web accessibility (target: 2.1 AA)
GDPR	General Data Protection Regulation — EU privacy framework applied as best practice
TLS	Transport Layer Security — cryptographic protocol securing all network communications
ACID	Atomicity, Consistency, Isolation, Durability — PostgreSQL transaction properties
UUID	Universally Unique Identifier — random 128-bit value used as primary key
JSONB	PostgreSQL binary JSON type — indexed, queryable flexible data storage
CSP	Content Security Policy — HTTP header restricting resource origins to prevent XSS
HSTS	HTTP Strict Transport Security — forces HTTPS for all future connections
DFD	Data Flow Diagram — shows how data moves between processes and data stores
RTO	Recovery Time Objective — maximum acceptable downtime (4 hours for critical functions)
RPO	Recovery Point Objective — maximum acceptable data loss (1 hour)
MFA	Multi-Factor Authentication — required for all administrator accounts
CDN	Content Delivery Network — distributes static assets geographically for faster load times

Appendix A: Use Case Diagrams

A.1 Test Taker Use Cases

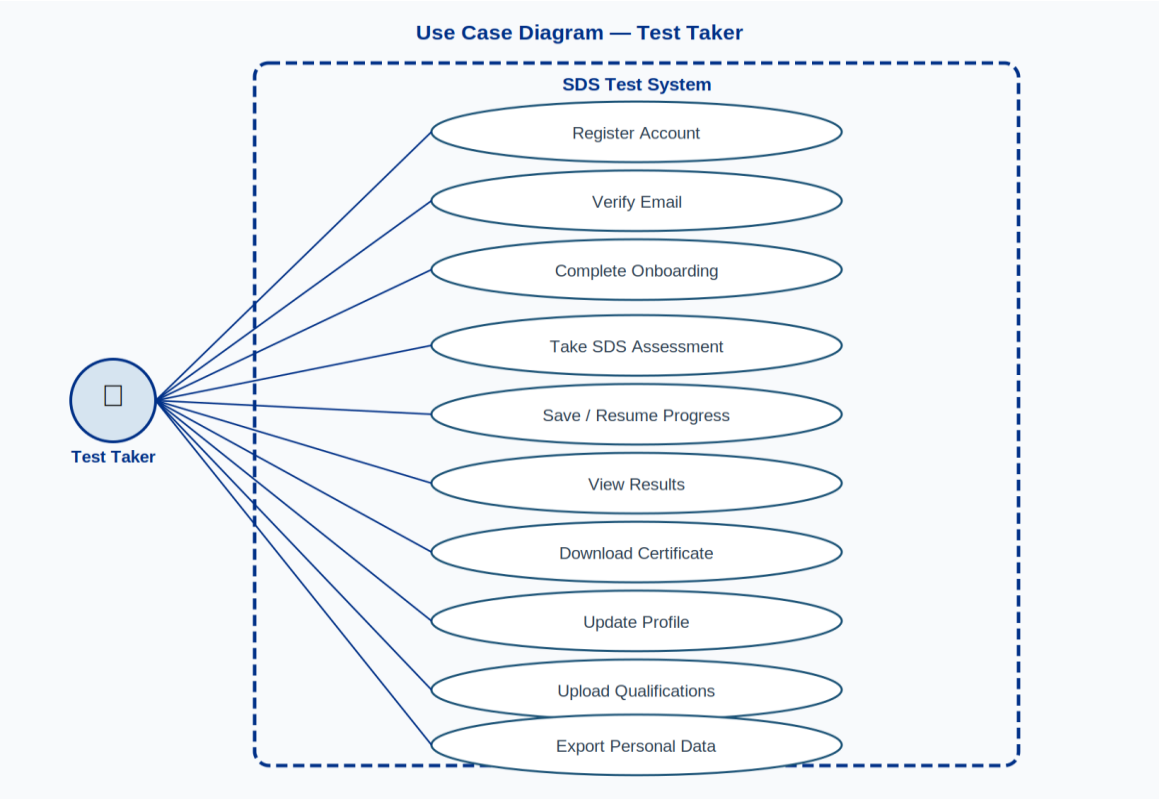

Figure A1: Use Case Diagram — Test Taker Role

A.2 Test Administrator Use Cases

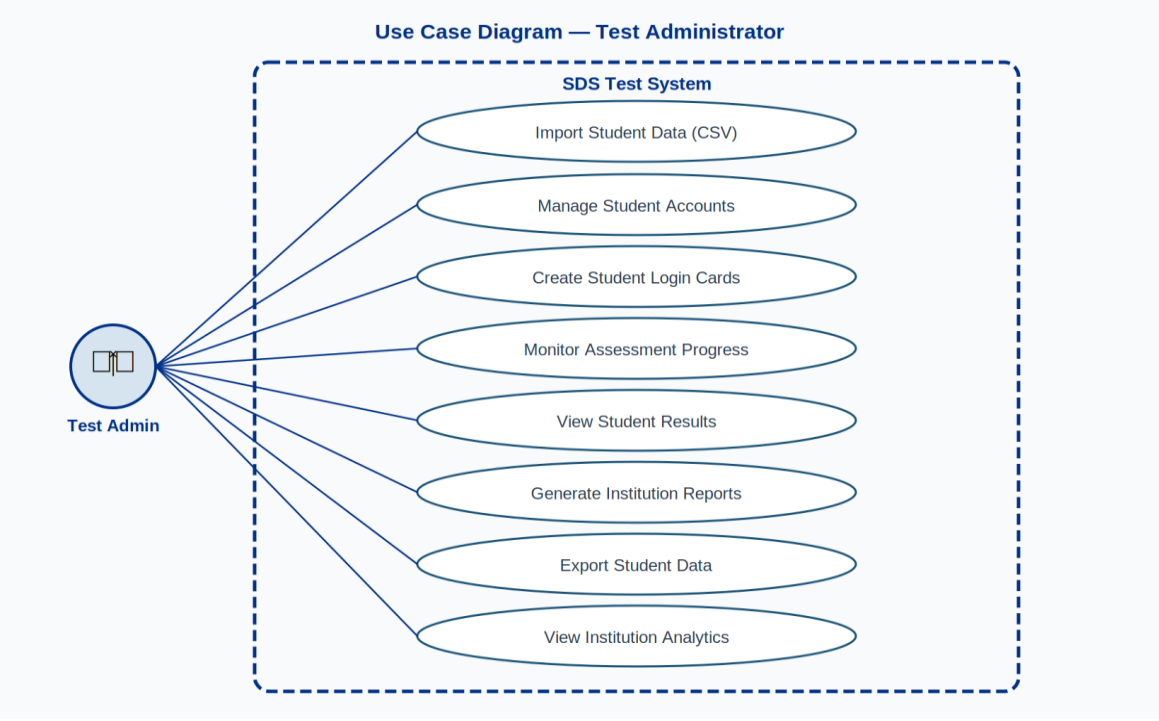

Figure A2: Use Case Diagram — Test Administrator Role

A.3 System Administrator Use Cases

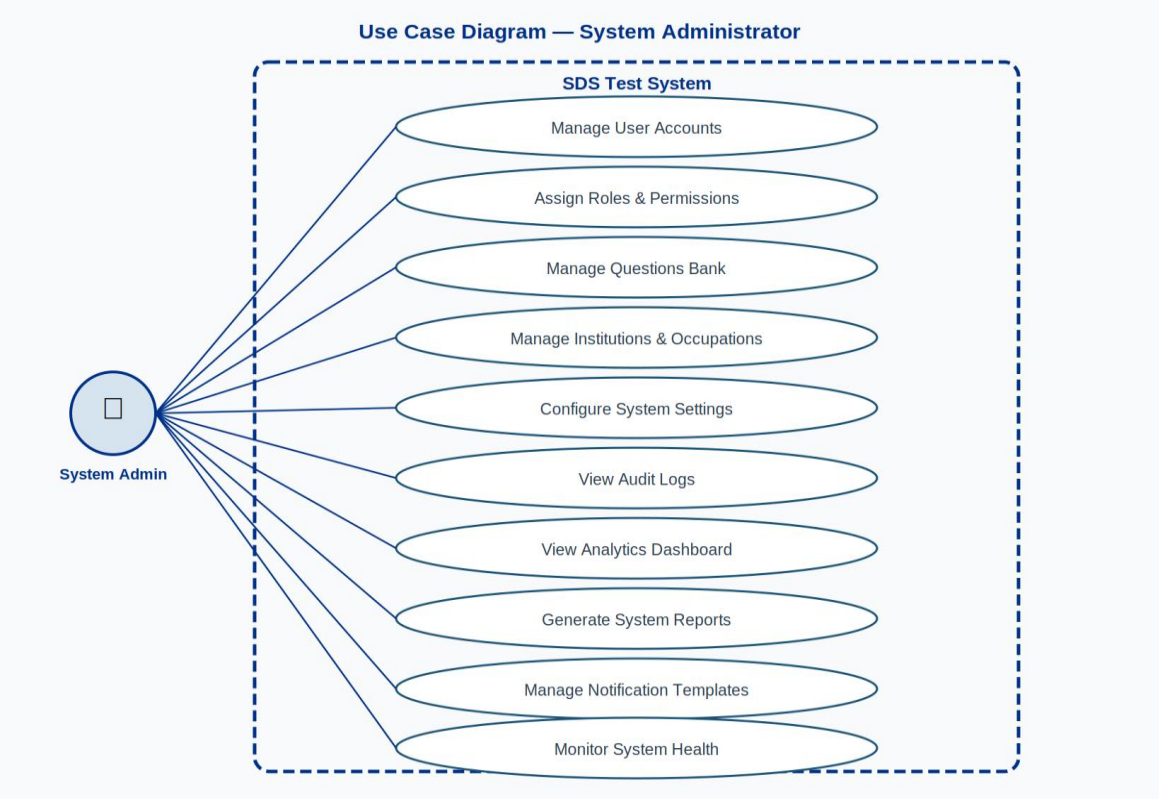

Figure A3: Use Case Diagram — System Administrator Role

Appendix B: Sequence Diagrams

B.1 User Login

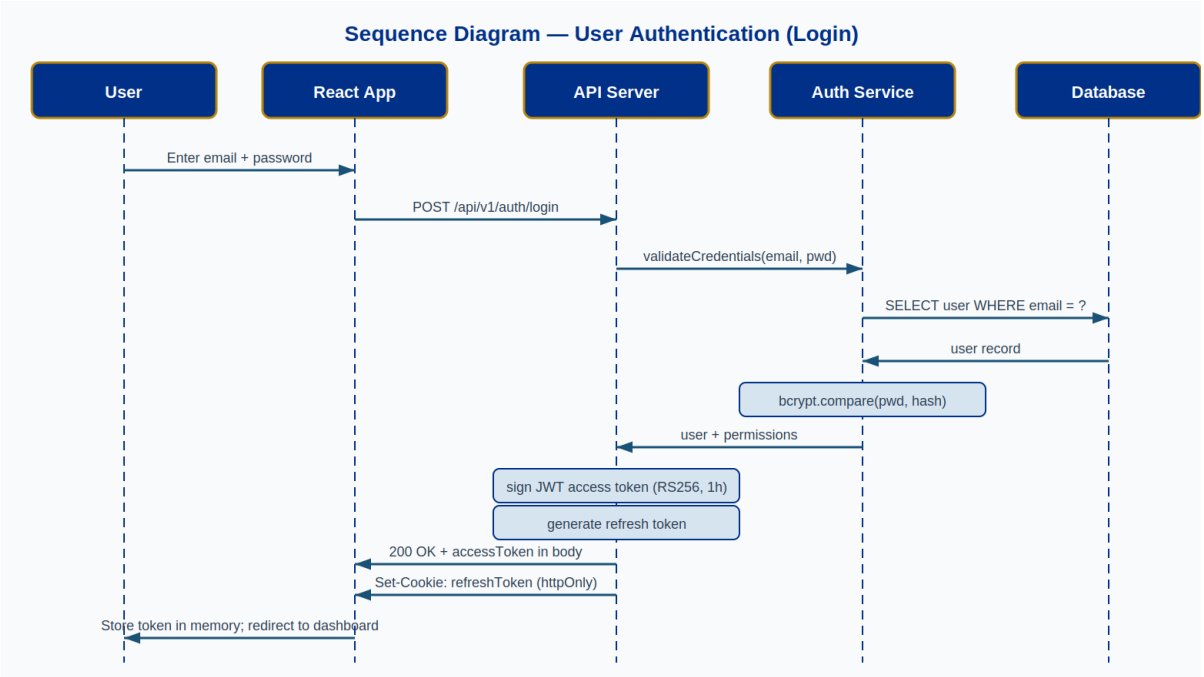

Figure B1: Sequence Diagram — User Authentication (Login)

B.2 User Registration & Email Verification

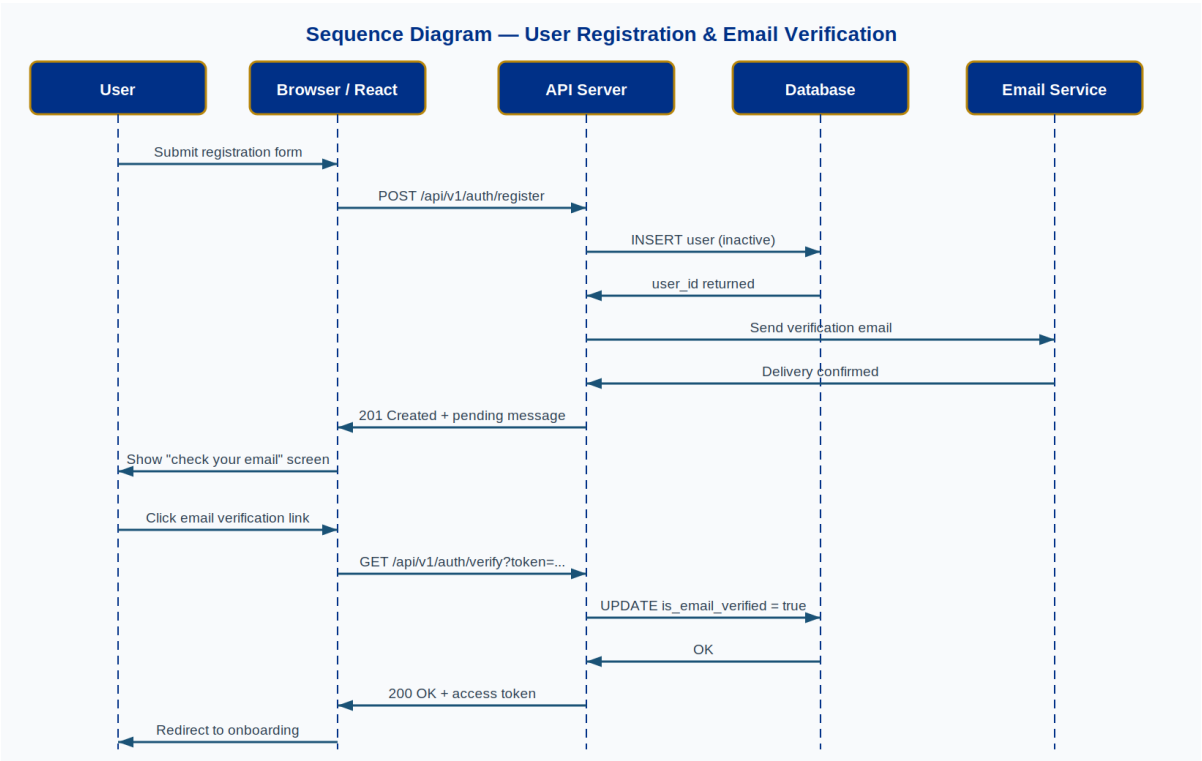

Figure B2: Sequence Diagram — Registration & Email Verification

B.3 SDS Assessment Completion

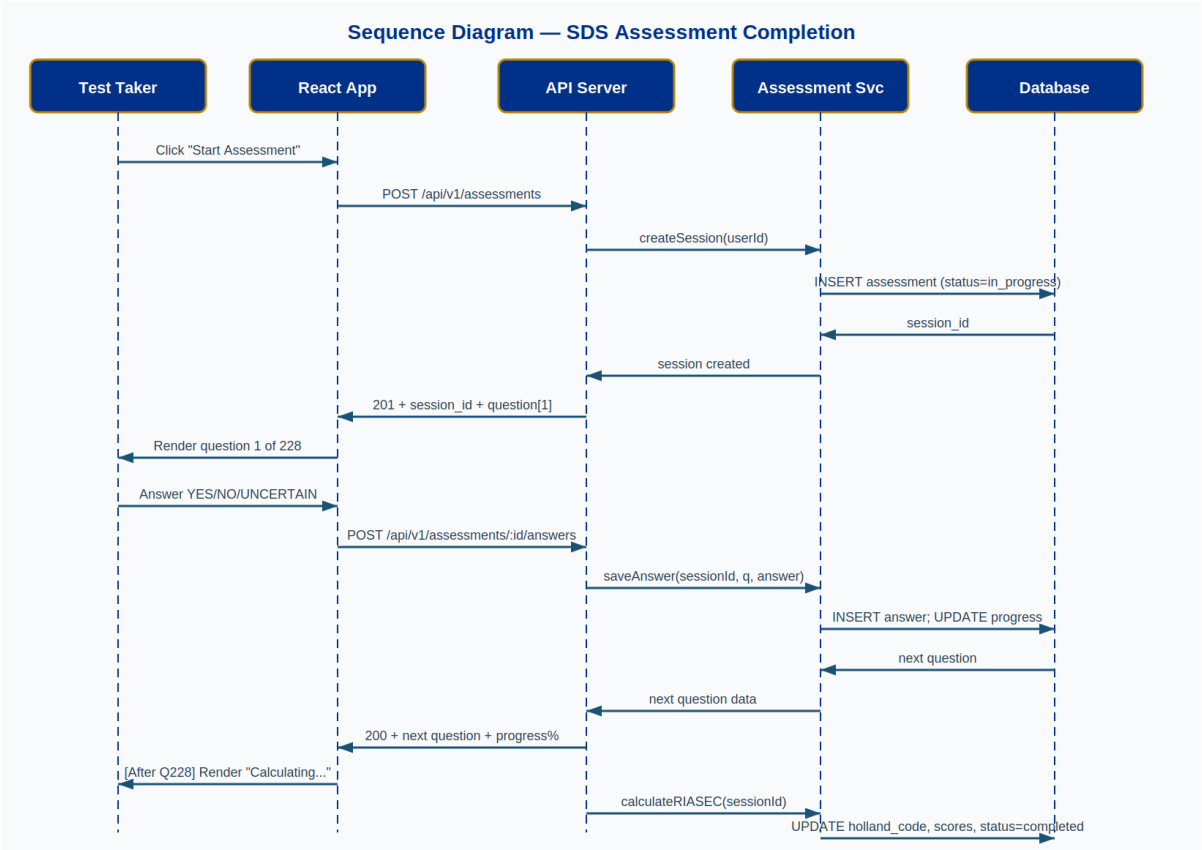

Figure B3: Sequence Diagram — Assessment Completion Flow

Appendix C: Activity Diagrams

C.1 User Registration Process

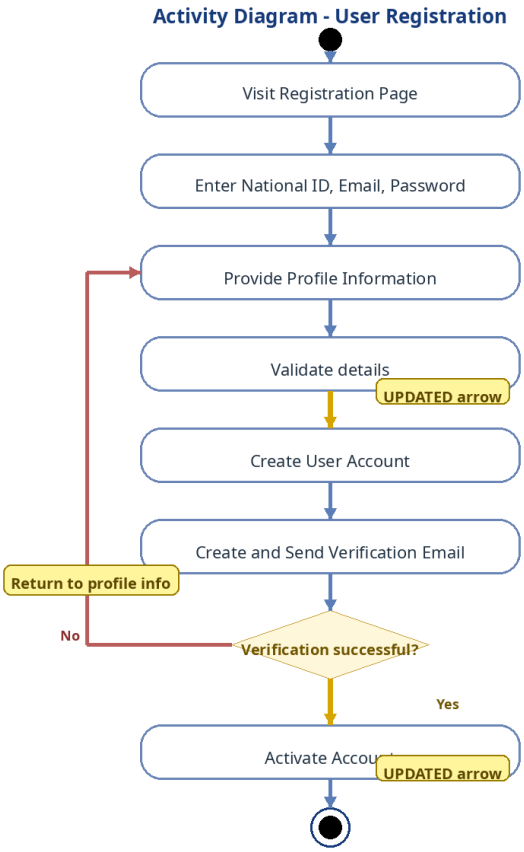

Figure C1: Activity Diagram — User Registration

C.2 Assessment Completion Process

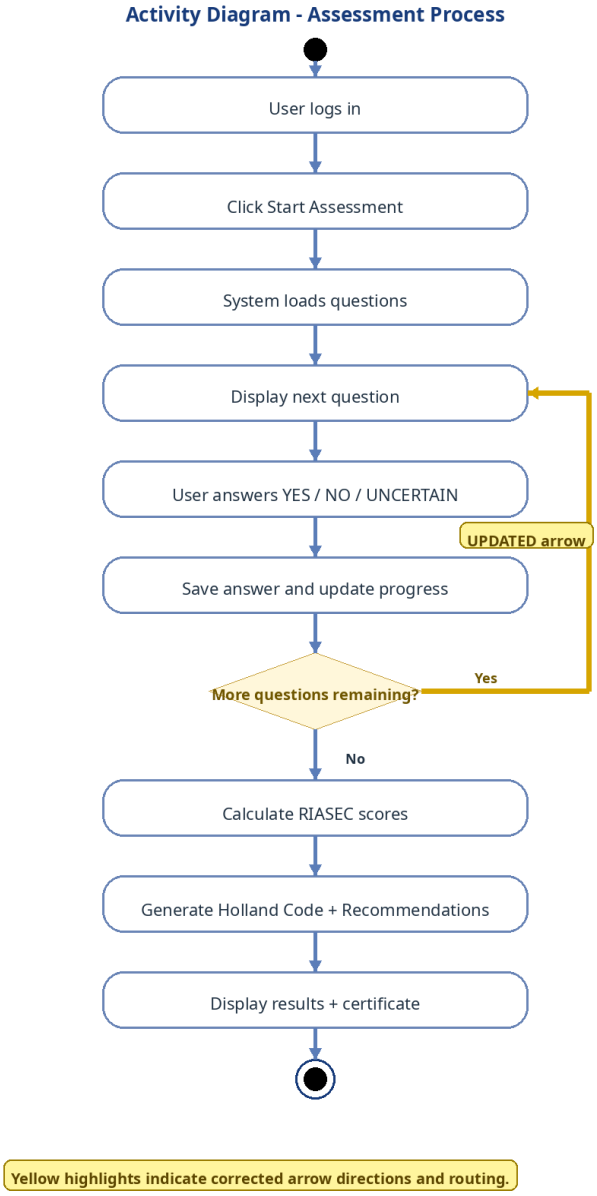

Figure C2: Activity Diagram — Assessment Process

Appendix D: State Machine Diagram

D.1 Assessment Session Lifecycle

The state machine below shows all possible states of an assessment session and the transitions between them:

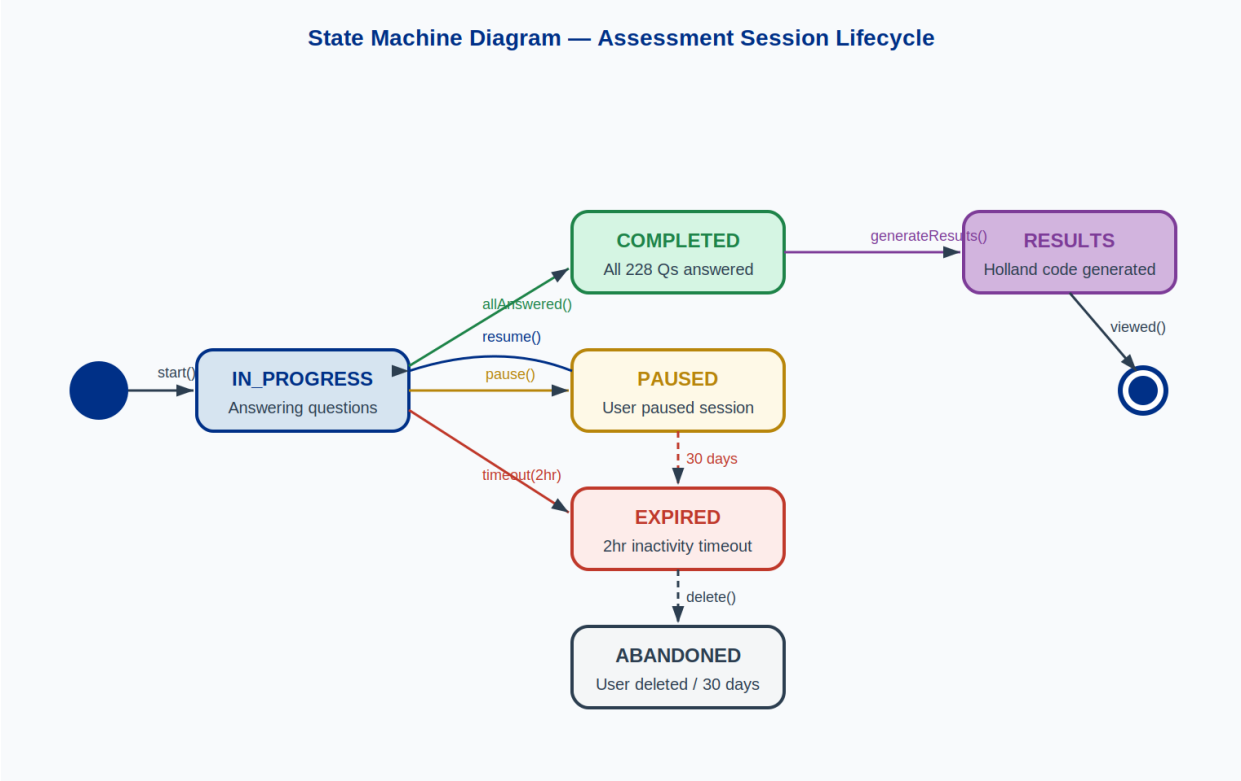

Figure D1: State Machine — Assessment Session Lifecycle

States and their meanings:

State	Description	Entry Condition	Exit Transitions
IN_PROGRESS	User is actively answering questions	User clicks 'Start Assessment'	→ PAUSED (pause), → COMPLETED (all 228 answered), → EXPIRED (2hr timeout)
PAUSED	User has saved progress and exited	User clicks 'Save & Pause'	→ IN_PROGRESS (resume within 30 days), → EXPIRED (30-day timeout)
COMPLETED	All 228 questions answered	Final answer submitted	→ RESULTS (generateResults() called)
EXPIRED	Session timeout occurred	2hr inactivity or 30-day pause limit	→ ABANDONED (user deletes or admin purges)
ABANDONED	Session permanently closed	User deletes or admin purges	Terminal state — no further transitions
RESULTS	Holland code generated and visible	calculateRIASEC() completes	Terminal state — certificate available for download

Appendix E: Component & Deployment Diagrams

E.1 Component Diagram

Detailed view of the three software packages and their internal components and interfaces:

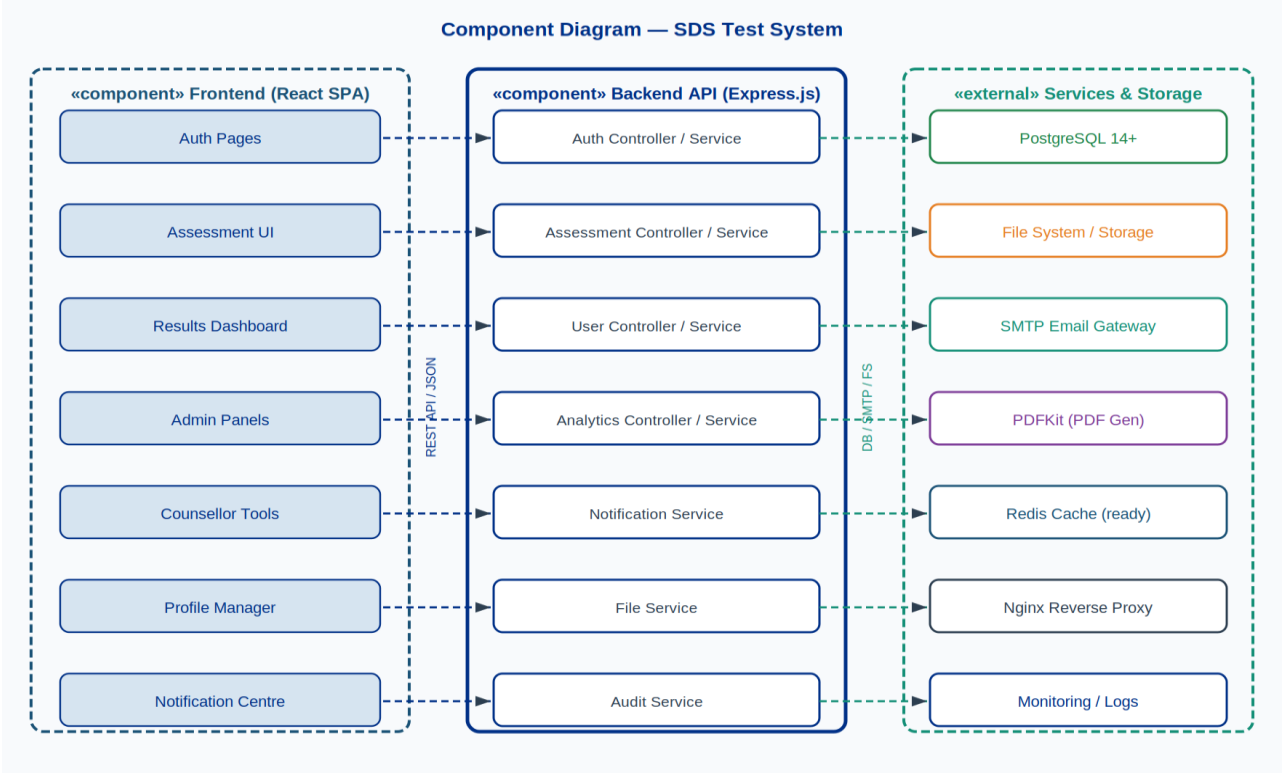

Figure E1: Component Diagram

E.2 Deployment Diagram

Physical infrastructure topology showing servers, containers, and network connections:

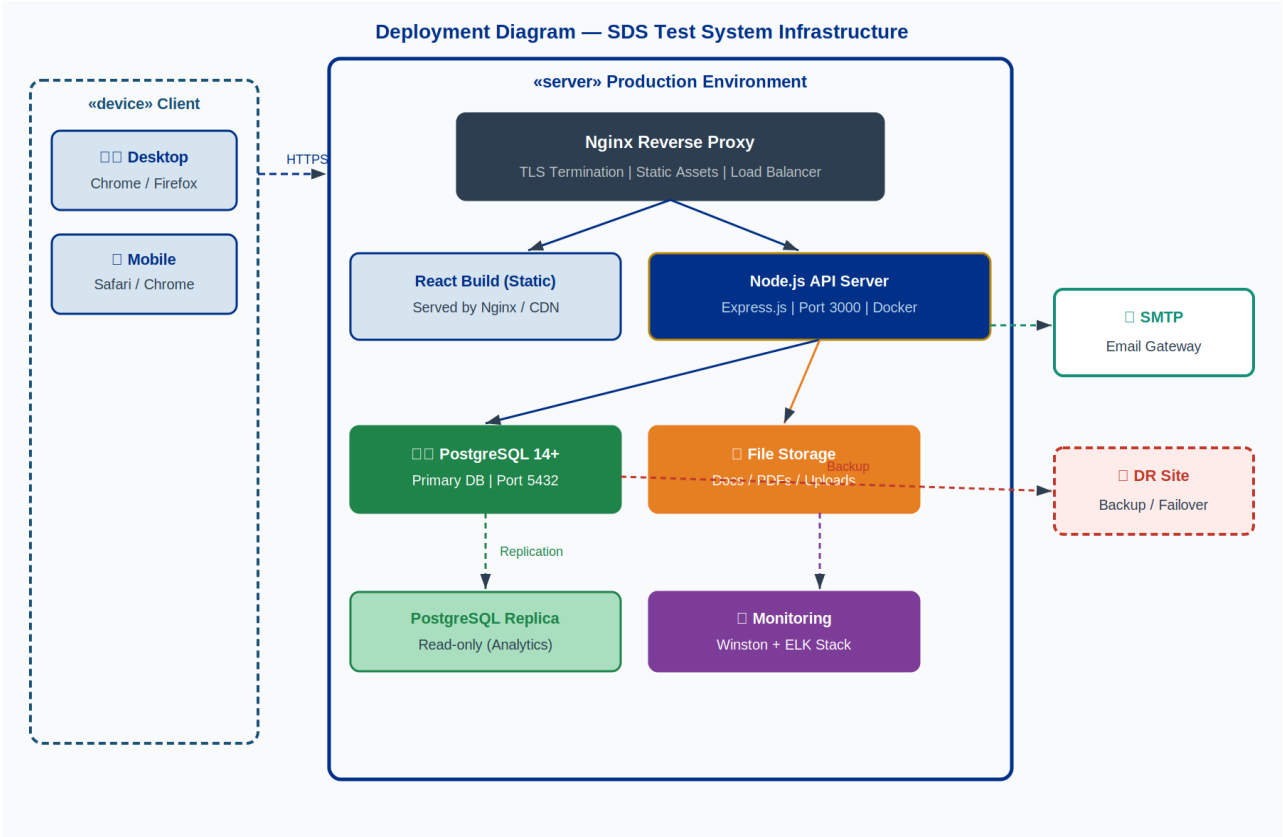

Figure E2: Deployment Diagram — Production Infrastructure

Appendix F: Class & ER Diagrams

F.1 Class Diagram

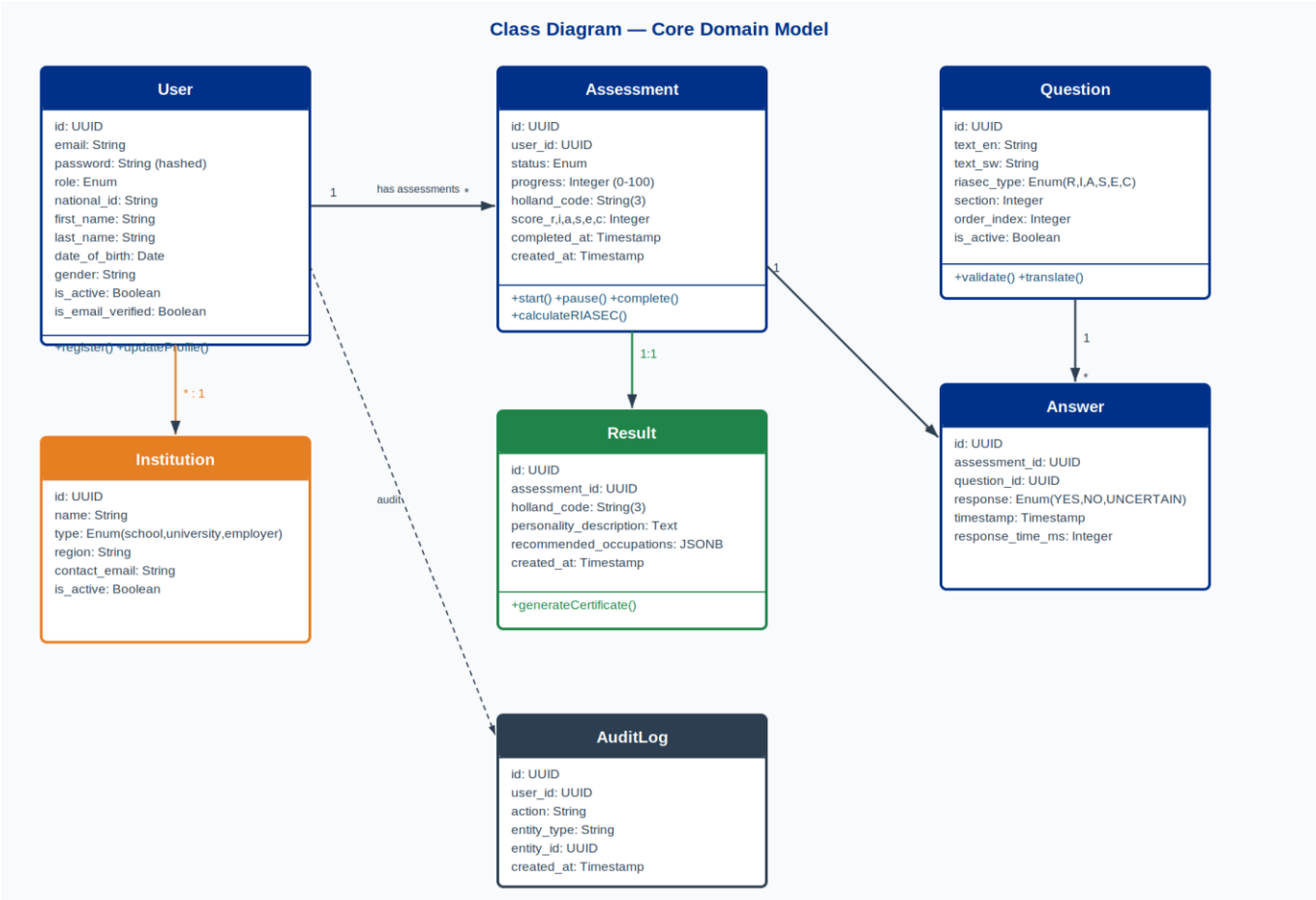

Figure F1: Class Diagram — Core Domain Model

F.2 Entity-Relationship Diagram

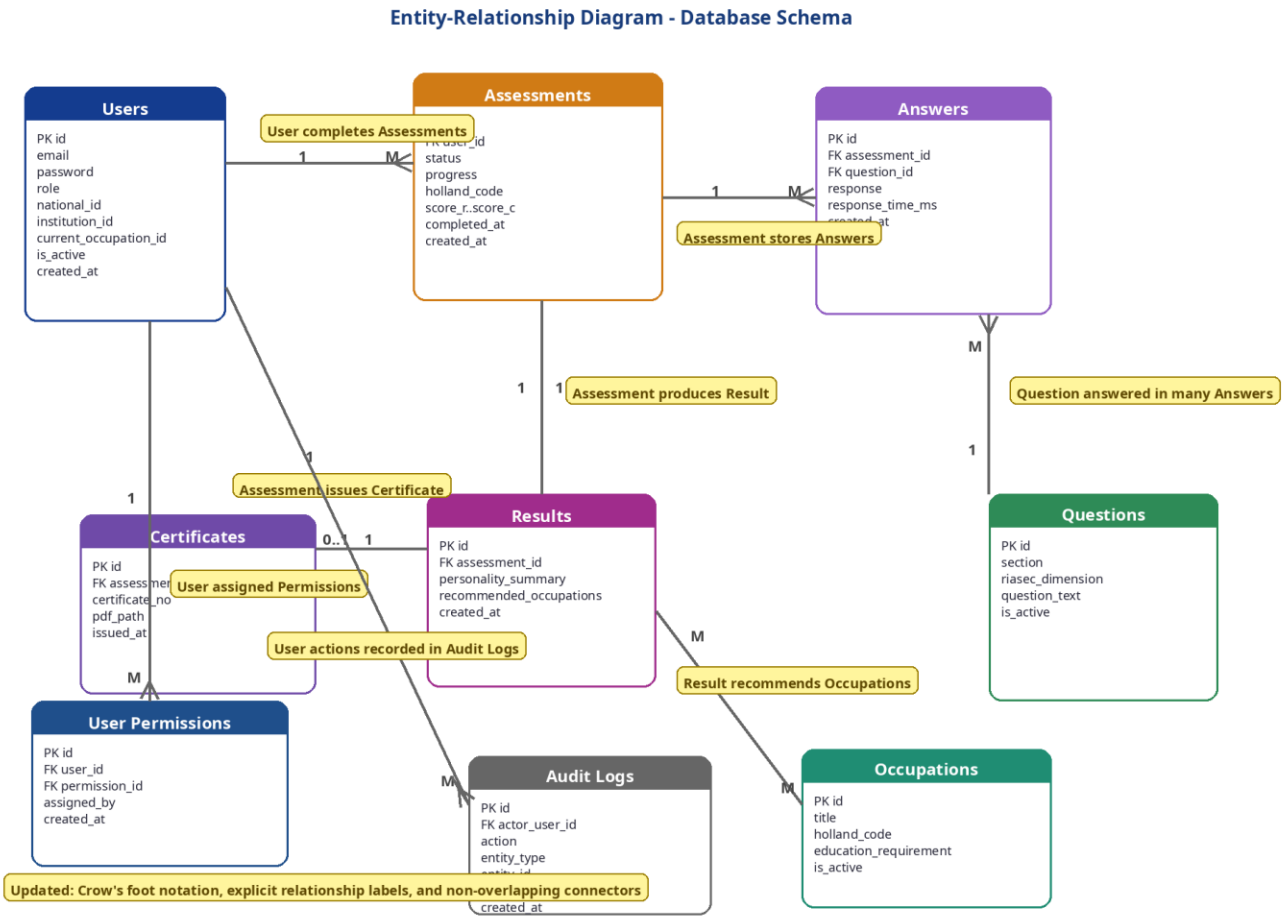

Figure F2: Entity-Relationship Diagram — Full Database Schema

Appendix G: Algorithm & Infrastructure Diagrams

G.1 RIASEC Scoring Algorithm

The diagram below illustrates the step-by-step algorithm used to calculate RIASEC scores and derive the Holland code from a completed assessment:

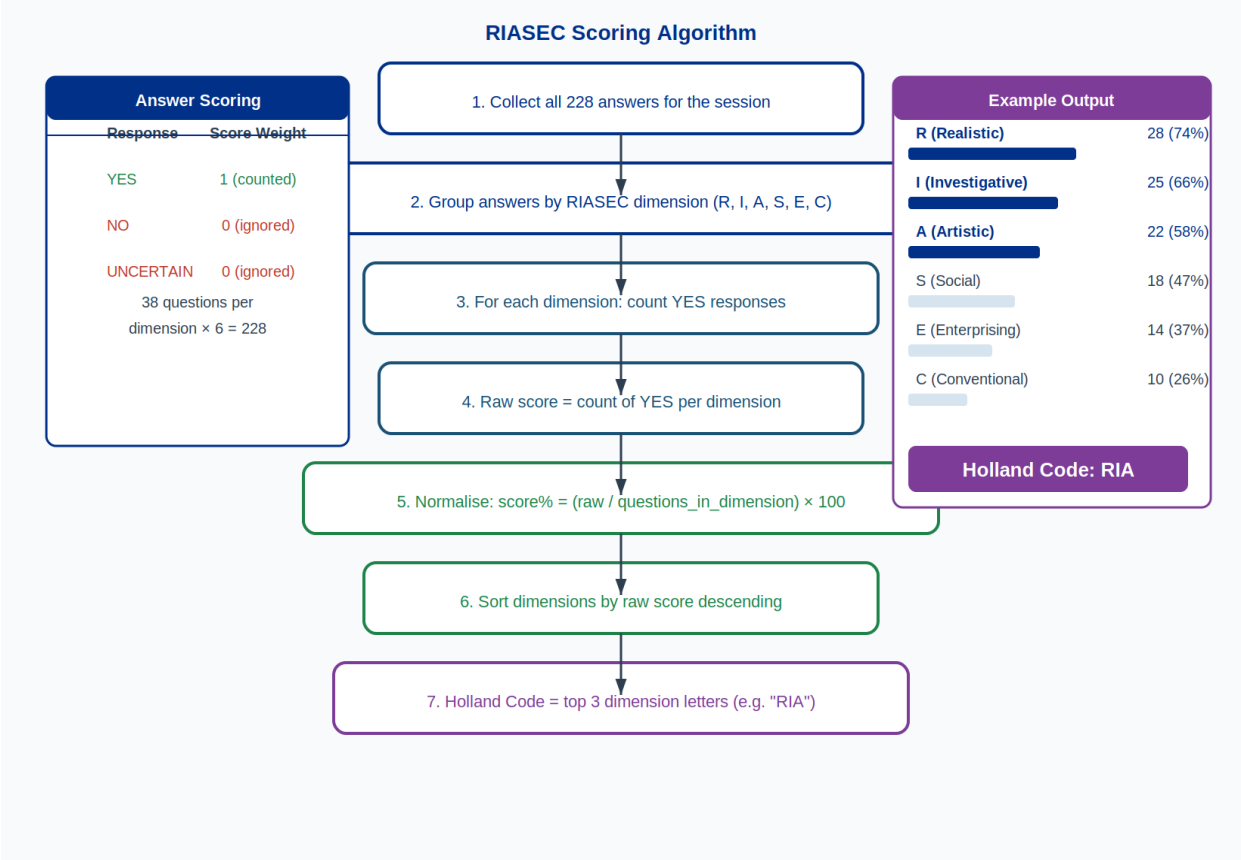

Figure G1: RIASEC Scoring Algorithm

G.2 RBAC Permission Visualisation

Which roles have access to which permission modules:

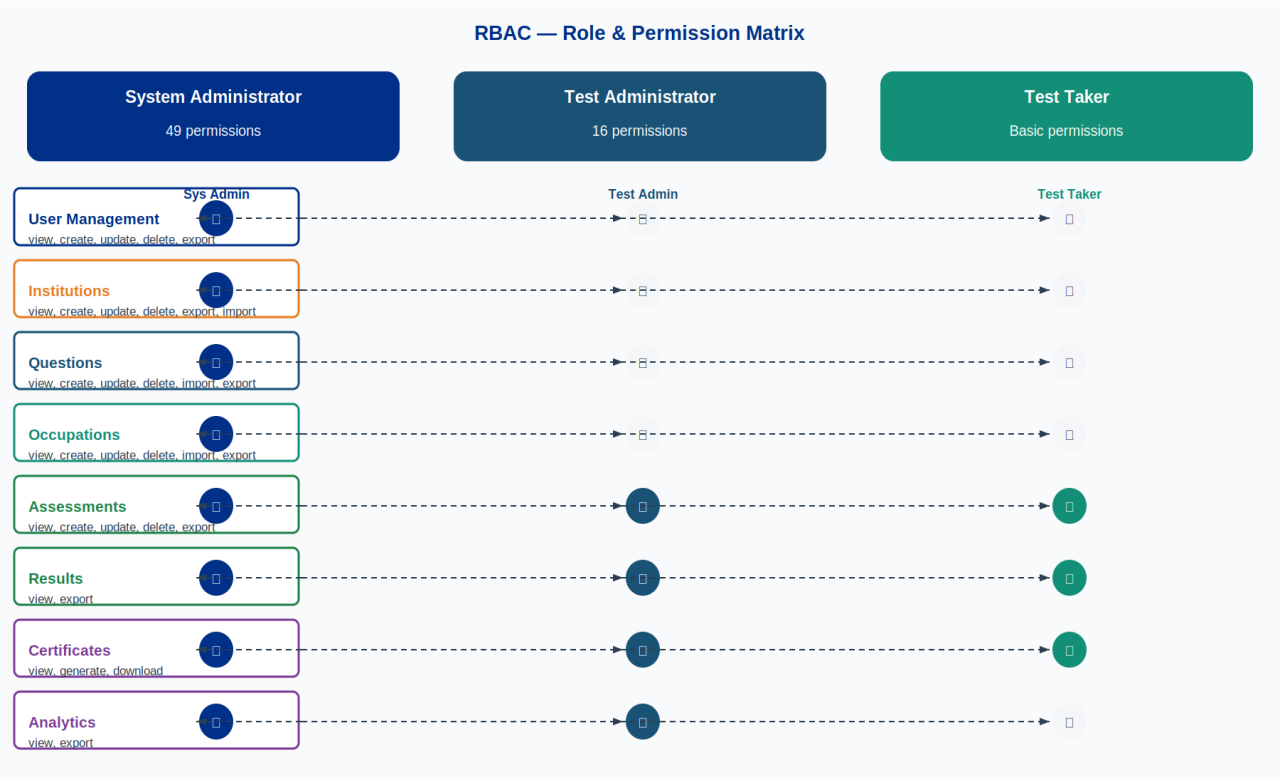

Figure G2: RBAC Role & Permission Matrix

Appendix H: UI Wireframes

> *These wireframes illustrate the intended user interface design. Final visual styling will follow the Ministry of Labour and Social Security brand guidelines. Wireframes are indicative of layout and interaction patterns.*

H.1 Assessment Screen

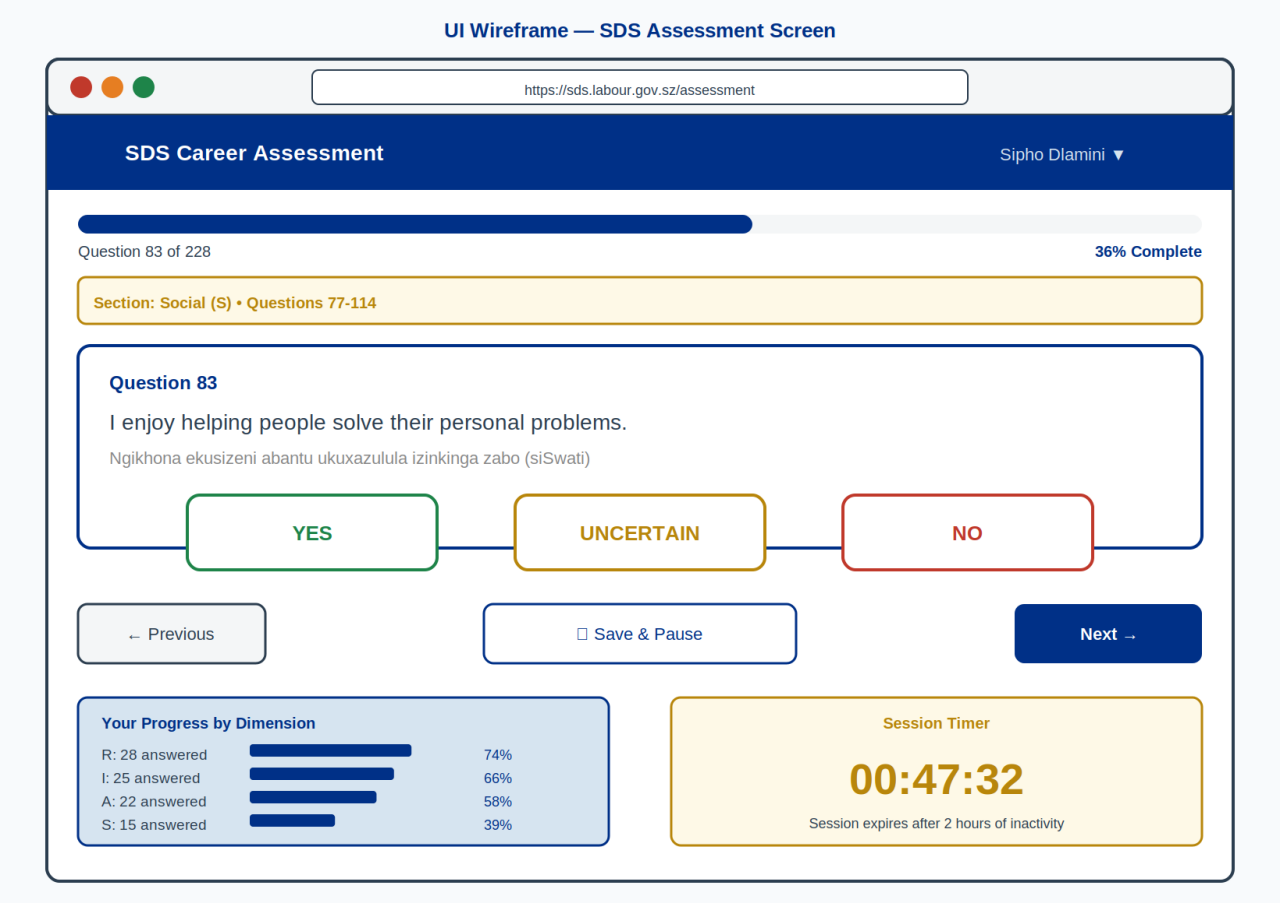

Figure H1: Wireframe — SDS Assessment Question Screen

H.2 Results & Recommendations Dashboard

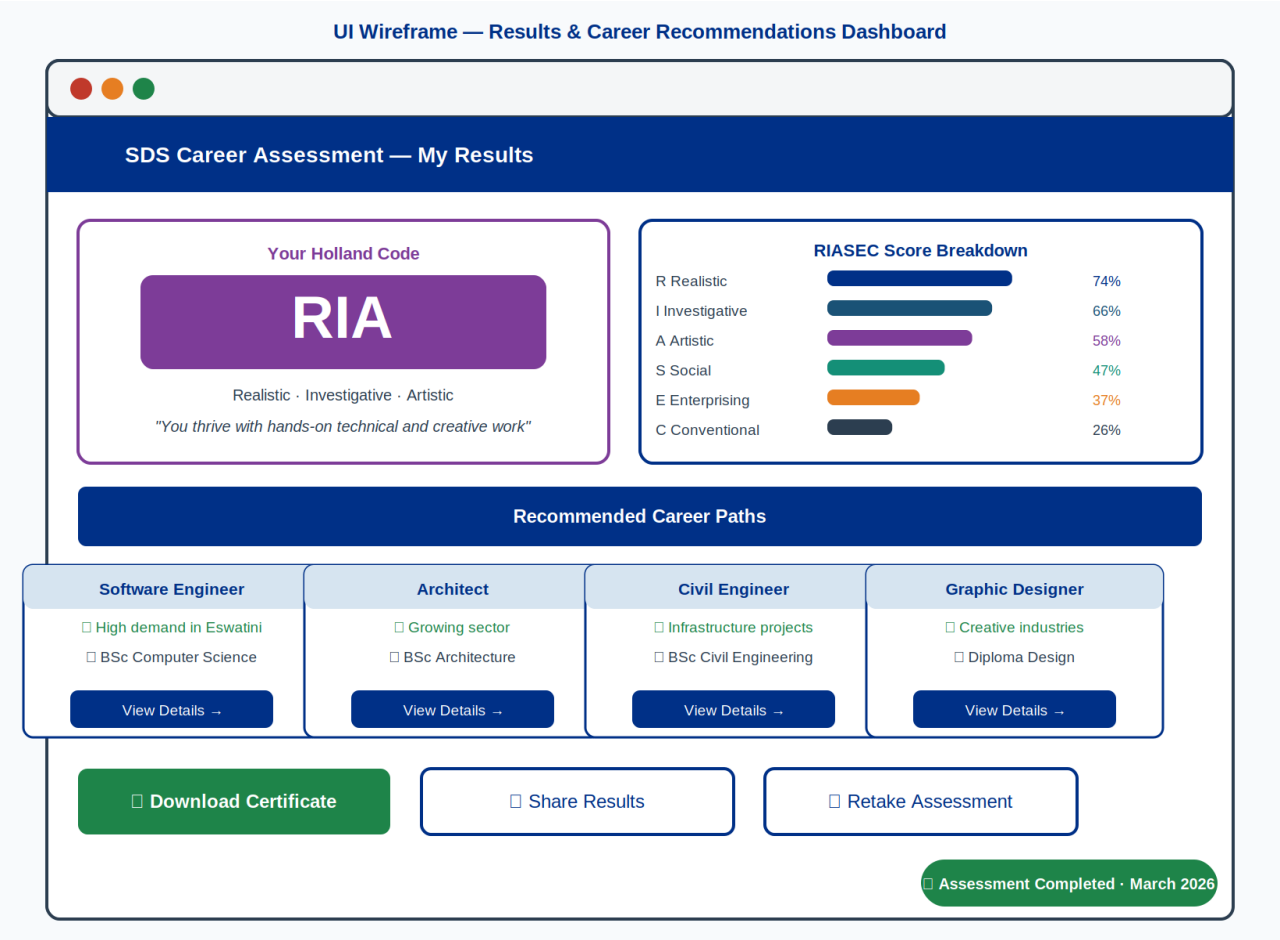

Figure H2: Wireframe — Results & Career Recommendations Dashboard

Appendix I: Document Approval

The following individuals are required to review and formally approve this System Design Document before implementation commences.

Role	Name	Signature	Date
Project Sponsor			
Technical Lead			
Quality Assurance Lead			
Business Analyst			
Ministry IT Representative			